

What's an algorithm?

→ Set of instructions to follow (Operational Definition)

≡ Time : Angle by which seconds hand moves

Actual definition → Causes revolution

- What's a computer?
- Separate Hardware / Software
- Programming Language
- Computer
- What's a machine?

Truth : Algorithm (Turing Algorithm)

→ Axioms (Assumptions) :

1) Information travels at a finite speed [Sp. Th. of relativity]

Loophole : RAM on Earth $\Rightarrow$ malloc array in outer space

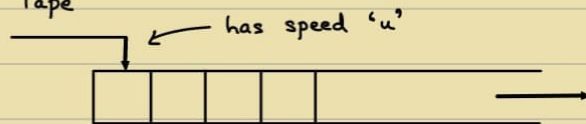
→ (Printing array[100] or array[10¹⁰⁰])

Every memory is sequential $\Rightarrow$ Memory $\equiv$ Tape

Only Sequentially accessible

Read - Write Memory

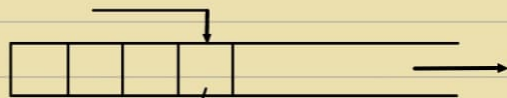
W.L.G One cell at a time (OR) can stay



2) Finite volume in space can be used to reliably store/retrieve only finite amount of information. [Quantum Mech.]

e.g. Pendrive

Loophole : π

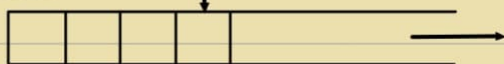


→ Finite set of Tape Symbols $\{\epsilon, T\}$

3) Only finitely many instructions can be 'packed' at a time



$q_i \in \mathcal{Q}$: Finite set of states



Turing Machine

A T.M is a 7 tuple

$$\langle Q, T, \delta, q_{start}, q_{accept}, q_{reject} \rangle \quad \delta: T \times Q \rightarrow T \times Q \times \{L, R\}$$

7th: Only occurs in Tape but not in Input

↳ Σ : finite set of input symbols, $\Sigma \subseteq T$

$$b \in T, b \notin \Sigma$$

$$\langle Q, T, \Sigma, \delta, q_{start}, q_{accept}, q_{reject} \rangle$$

ALGORITHM: Simulatable in some TM (Church-Turing Thesis / Hypothesis)

Computer: Algorithm

↳ Realistic machines

↳ Device which follows the axioms

VR: Anything that is real can also be virtual.

Single-structure programming Language ← Turing Machine

Note: Turing Machine can be input to TM.

$$\bullet U_{TM}(\langle M \rangle, x) = M(x)$$

Universal Turing Machine: Universal General Purpose Software

x : Description of Circuit

M : Software

Software: Written on top of hardware

WHY:

$$\left. \begin{array}{l} F_i = F_{i-1} + F_{i-2} \\ F(1) = 1 \\ F(0) = 0 \end{array} \right\} \text{Recursion}$$

Memoization } Iteration

$$F_n = \frac{1}{\sqrt{5}} (\phi)^n - \frac{1}{\sqrt{5}} (\bar{\phi})^n \quad \left. \vphantom{F_n} \right\} \text{Formula}$$

$$\left[\phi = \frac{1+\sqrt{5}}{2} \right]$$

↳ Eigenvalues [Hint to solve]

How:

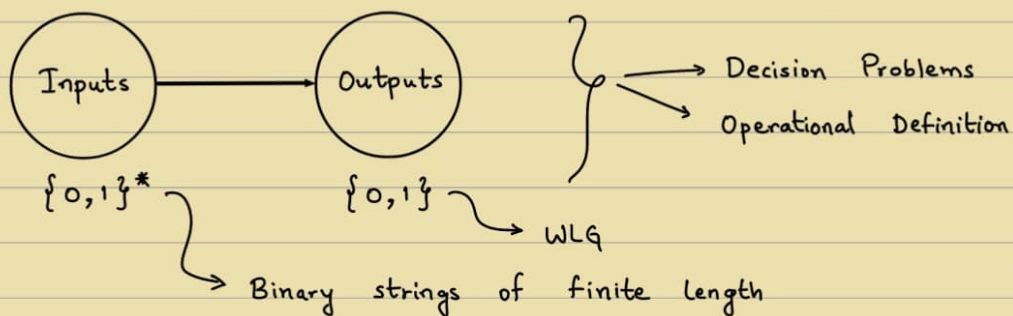
Algorithms book by Dasgupta, Vazirani

- Divide and Conquer
- Greedy Algorithm
- Dynamic Programming
- Linear Programming
- Complexity Theory
(Hardness)
- Coping with Hard Problems
(Random, Approx, Quantum, etc)
- Unsolvable problems

4/8

Q) WAP to _____
 ↳ Write A Program

→ What are computation problems?



Output $\rightarrow \{0,1\}$
 $\therefore$ If Output > 1 bit $\Rightarrow$ Encode into multiple problems
i.e. if 01 $\rightarrow$ solution
0 : Solution of Problem 1 [2^1 place]
1 : Solution of Problem 2 [2^0 place]

No output $\Rightarrow$ Not a computation problem (Acc. to course)

$\therefore$ Decision Problem $\rightarrow$ 2-Partition of the input set
 ↳ Language



Language $L \subseteq \{0,1\}^*$

Note: Decision Problem $\equiv$ Membership queries in Languages

Q) Given a graph, is it connected or not?

Graph $\rightarrow$ Input

$\rightarrow$ Encode into $\{0,1\}^*$, i.e. Adjacency Matrix/List

[OR] $G \in \{G' \mid G' \text{ is a connected graph}\}$

Q) Given a natural no., is it prime?

[OR] $n \in \{i \mid i \text{ is prime}\}$

Note: If Answer is True/False $\Rightarrow$ Always decision problem



Always computation problem

[e.g. Membership problem]

Q) WAP to print smallest prime no. $\equiv$ WAP to print smallest even no.?

YES in formal language

(NO in English)

[Can you formalize it or not?] $\leftarrow$ Bigger Question

$\rightarrow$ Computation problem or not?

• e.g. LLMs: Speech to text C program

$\rightarrow$ Creates problems rather than solving it lol
when we don't define the problem

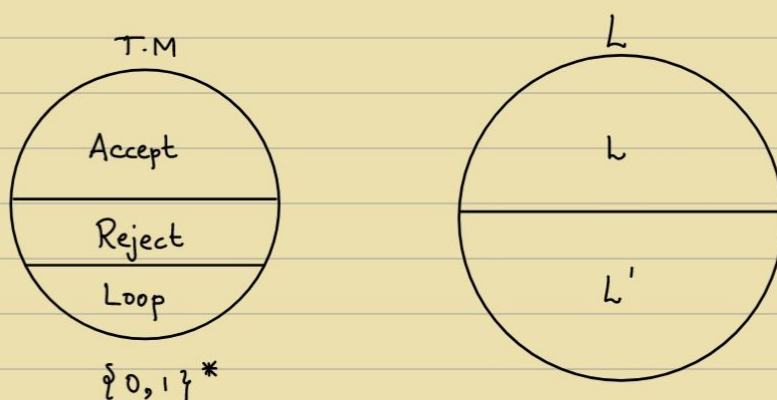
$\rightarrow$ Poses non-CSE problems as CSE problems

* $\left\{ \begin{array}{l} \text{Given a language} \rightarrow \text{We have a computation problem at hand} \\ \text{Given a T.M} \rightarrow \text{We have an answer / solution / program at hand} \end{array} \right\}$

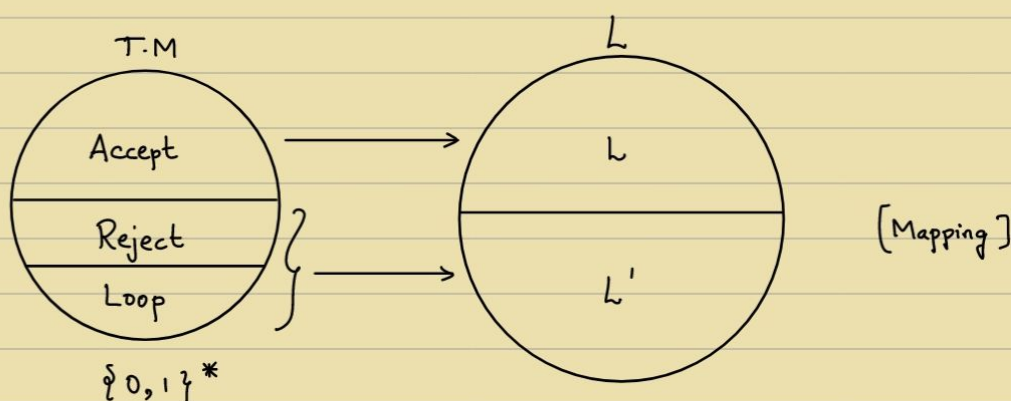
$\rightarrow$ What connects them?

→ When do we say that a program solves a problem?

e.g. $n \in \text{Primes}$?



Q) When do we say a T.M solves a problem?



Defⁿ: A language L is said to be recognized by T.M 'M' if
 ↳ Acc to world

$\forall x \in L \rightarrow M(x) \text{ accepts}$

$\forall x \notin L \rightarrow M(x) \text{ does not accept}$

$$L(M) = \{x \mid M(x) \text{ accepts}\}$$

Defⁿ: A language L is said to be decided by T.M 'M' if
 ↳ Acc to world

$\forall x \in L \rightarrow M(x) \text{ accepts}$

$\forall x \notin L \rightarrow M(x) \text{ rejects}$

"This one, ChatGPT will also not know"

- AAD Prof, 2025

{ Active voice is preferred in Scientific Computer Literature } ☹️

→ Towards Optimization :

- What are resources ?
- Quantifying their use
- Best Notions, etc

Computability Theory

What is a Theory ?

Note : When something is an art, that means not everyone can do it

Science : Every man's art [No talent required]

Gifted Scientist X

Scientist ✓

Theory gives you the answer to scientific problems.

when you can't perform scientific experiments.

Computability Theory : Tells what can be solved, what can't

Does $\exists$ a C program ?

Functionality Theory :

TM $\rightarrow$ works in steps, $\therefore$ Complexity can be measured

i.e. How much memory it takes ?

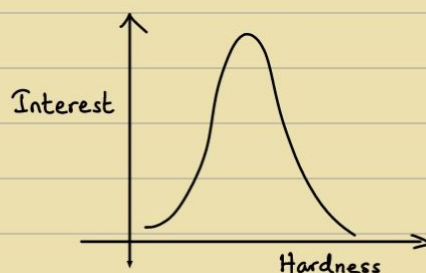
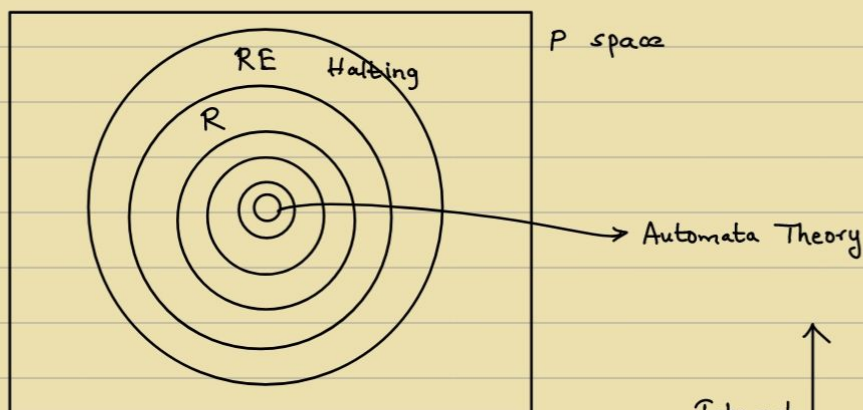
How much time it takes ?

$\rightarrow$ default

"Whenever there is a reality, there is a virtual reality"

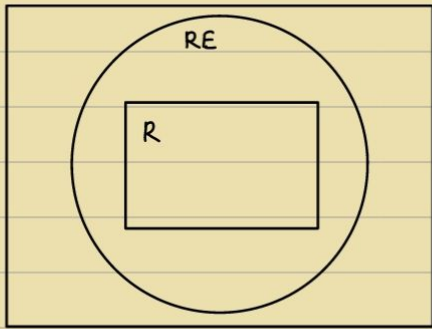
Theorem : There are Languages that are unrecognizable

Theorem : There are Languages that are recognisable but undecidable.



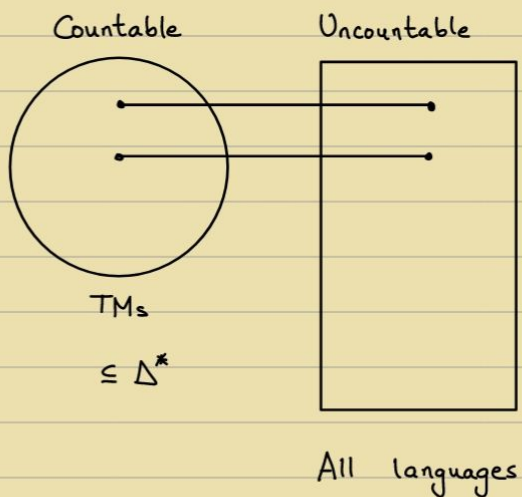
Defⁿ: The class of Recursively enumerable (RE) language is
 $RE = \{L \mid \exists \text{ T.M 'M' that recognises } L\}$

Defⁿ: The class of Recursive (R) language is
 $RE = \{L \mid \exists \text{ T.M 'M' that decides } L\}$



- Q) Are there languages unrecognisable by TMs / Computers / C-Programs ?
- Q) Are there undecidable languages ?

Proof.



Theorem: $\{0,1\}^*$ is countable

Proof: $f: \mathbb{N} \rightarrow \{0,1\}^*$

$\epsilon, 0, 1, 00, 01, 10, 11, \xleftrightarrow{8}, \xleftrightarrow{16}$

$f(1) = \epsilon$	$f(3) = 1$
$f(2) = 0$	$f(4) = 00$

Theorem: Set of all languages is uncountable.

Proof: A language $L \subseteq \{0,1\}^*$

Powerset of $\{0,1\}^* \rightarrow 2^{\{0,1\}^*}$

Every element in powerset can be uniquely described by a binary string.

$\therefore$ Bijection

e.g. $A = \{1, 2, 3\}$, $|A| = 3$

$P(A) = \{ \dots, \{2, 3\}, \dots \}$

$\swarrow$
0 1 1
 $\swarrow$ 1 is absent
 $\searrow$ 2 & 3 are present

On the contrary, Suppose
 $2^{\{0,1\}^*}$ is countable

~~$f(1) = b_{11} b_{12} \dots b_{1i} \dots$~~

~~$f(2) = b_{21} b_{22} \dots b_{2i} \dots$~~

~~$\vdots$~~

~~$f(i) = b_{i1} b_{i2} \dots b_{ii} \dots$~~

Method of diagonalisation :

$L = b_{11} b_{22} b_{33} \dots b_{ii} \dots$

$\swarrow$ Can't be $f(1) \because$ Differs at least 1 location from $f(1)$

Can't be $f(2) \because$ Differs at least 1 location from $f(2)$

Can't be $f(i) \because$ Differs at least 1 location from $f(i)$

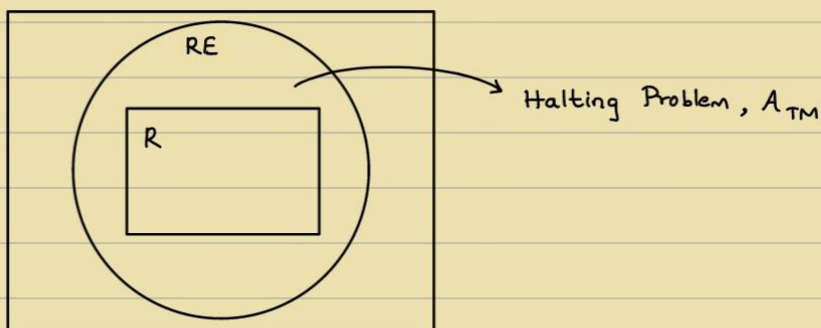
$\therefore$ Can't get bijection

$\therefore$ Can't be onto

$\therefore$ Uncountable

RE : Yes $\rightarrow$ Yes

No $\rightarrow$ No [OR] Unknown



- Given a TM 'M' and input 'x', does it accept x?

$$A_{TM} = \{ \langle M, x \rangle \mid M \text{ accepts } x \}$$

Yes $\rightarrow$ Accept

No $\rightarrow$ Reject [or] Run forever

$$\therefore A_{TM} \in R.E$$

$$A_{TM} \notin R$$

Theorem: A_{TM} is undecidable

Proof: Suppose A_{TM} is decidable

Let TM 'H' decide A_{TM}

$$\forall (M, x) \in A_{TM}, H(M, x) = \text{Accept}$$

$$\forall (M, x) \notin A_{TM}, H(M, x) = \text{Reject} \quad [H : \text{Halting TM}]$$

Consider a TM 'D'

On input $\langle M \rangle$

$\rightarrow$ Runs $H(M, \langle M \rangle)$

$\rightarrow$ If H accepts, REJECT

Else if H rejects, ACCEPT

$\langle M \rangle$: Typecast program 'M' as strings

Execute $D(\langle D \rangle)$

On input $\langle D \rangle$,

$\rightarrow$ Run $H(D, \langle D \rangle)$

$\rightarrow$ If H accepts, then REJECT

if D accepts $\langle D \rangle$, then REJECT $\leftarrow$ Which is impossible

$\rightarrow$ If H rejects, then ACCEPT

If D does not accept $\langle D \rangle$, then ACCEPT $\leftarrow$ Which is illogical

$\therefore D$ does not exist $\Rightarrow H$ does not exist

(OR) Proof by diagonalisation method

no. of TMs $\rightarrow$ countable

$\therefore$ Can be enumerable & orderable

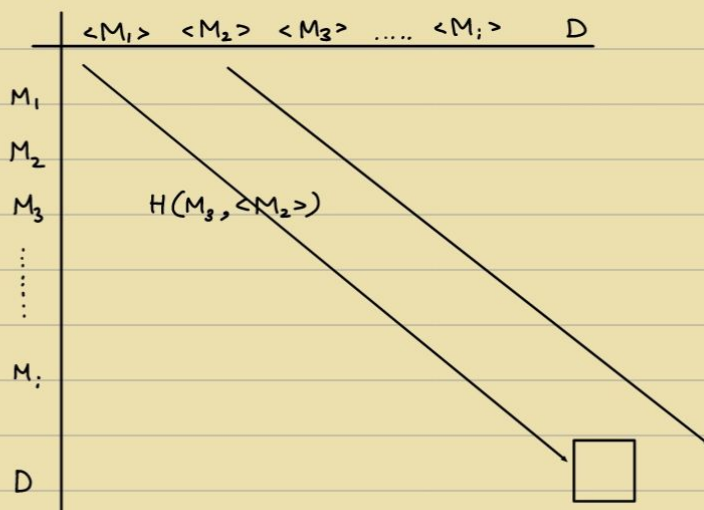


Table must have ACCEPT or REJECT.

But we get Blank
i.e. Whatever in diagonal,
do opposite
 $\therefore$ H cannot decide on
all inputs

Theorem: If $L \in RE$, $\bar{L} \in RE$, then $L \in R$

Proof: $L \in RE \exists$ TM 'M' s.t

$\forall x \in L \Rightarrow M$ accepts x

$\forall x \in \bar{L} \Rightarrow M$ does not accept x

$\bar{L} \in RE \exists$ TM 'M' s.t

$\forall x \in \bar{L} \Rightarrow M'$ accepts x

$\forall x \in L \Rightarrow M'$ does not accept x

Decider for L :

Just run M and M' in parallel

Theorem: $\bar{A}_{TM} \notin RE$

Proof: $A_{TM} \in RE$

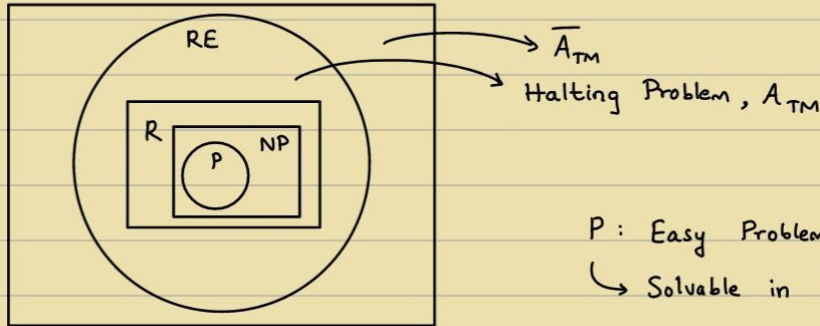
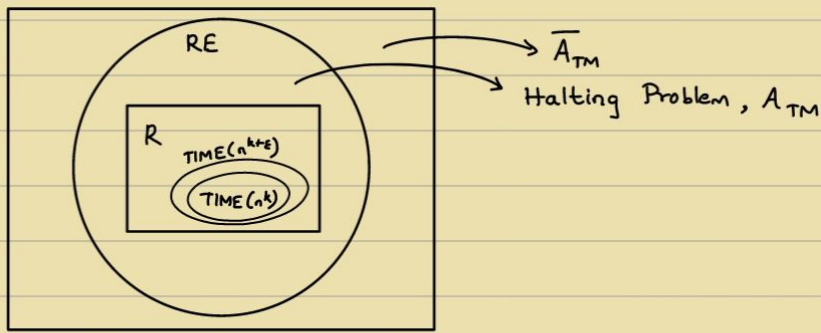
$A_{TM} \notin R$

$\} \Rightarrow \therefore \bar{A}_{TM} \notin RE$

$\rightarrow$ Take a class $O(n^k) \equiv TIME(n^k)$

Take another class $O(n^{k+\epsilon}) \equiv TIME(n^{k+\epsilon})$

For what value of ϵ , will they be diff.?



P: Easy Problems

↳ Solvable in Polynomial time

• Reduction: (Technique)

Problem A } Given
Problem B }

Use Problem B as a subroutine to solve Problem A
(Given there is a program for B.)

Defⁿ: A language is mapping-reduced to L_2 if

$\exists$ computable function f , $f: \{0,1\}^* \rightarrow \{0,1\}^*$ s.t.

$$\forall x \in \{0,1\}^*, x \in L_1 \iff f(x) \in L_2$$

Denoted as $L_1 \leq_m L_2$

11/8

The chalkboard contains the following handwritten notes:

- Review**
 - $\rightarrow A_m \in \{0,1\}^*$
 - $\rightarrow A_m \in RE, A_m \notin R$
 - $\rightarrow$ Proof by diagonalization
 - $\rightarrow A_m \notin RE$
 - $\rightarrow \exists L \in RE, L \notin R$
 - $\rightarrow$ All $\leq_m$ TM
 - $\rightarrow$ Decision Problem = Language
 - $\rightarrow$ TM is program language $L \in L$
 - $\rightarrow$ TM M decides L if it accepts L
 - $\rightarrow$ Given RE and R
- Reduction**
 - Mapping or Many-to-One Reduction
 - Def: Language $A \leq_m B$ if $\exists$ computable f s.t. $\forall x \in \{0,1\}^*, x \in A \iff f(x) \in B$
- Example**
 - Let $HALT_m = \{ \langle M, x \rangle \mid \text{TM } M \text{ halts on input } x \}$
 - Lemma: $A_m \leq_m HALT_m$
 - Proof: Decider for A_m (using one for $HALT_m$)
 - Else if H accepts: $\rightarrow$ Run M on x
 - $\rightarrow$ if M accepts, ACCEPT
 - $\rightarrow$ if M rejects, REJECT
- Theorem**
 - If $A \leq_m B$ and $B \in R$ then $A \in R$.
 - If $A \leq_m B$ and $A \notin R$ then $B \notin R$.

→ Completeness Theory:

Theorem: Halt_{TM} is RE-complete

Suppose $\forall A \in \text{RE}, A_M \leq \text{Halt}_{\text{TM}}$,

Then, Halt_{TM} is RE-complete

Proof:

Let TM 'M' recognise A &

H decides Halt_{TM}

Decider for A:

On input x , Run $H(M, x)$ if H reads REJECT

Else, Run M on x and do linking

→ $\text{EQUAL}_{\text{TM}} = \{ \langle M_1, M_2 \rangle \mid L(M_1) = L(M_2) \}$

Theorem: EQUAL_{TM} is unrecognisable

Proof: $\overline{A_{\text{TM}}}$ is unrecognizable ($\overline{A_{\text{TM}}} \notin \text{RE}$, known)

$M, x \in \overline{A_{\text{TM}}}$

Let E recognise EQUAL_{TM}

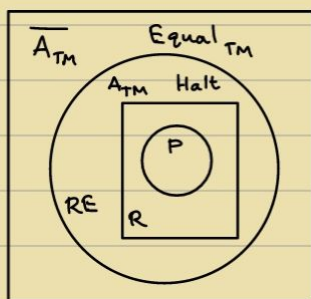
M_1 : On input w , REJECT

M_2 : On input w , if $x \notin w$, REJECT

if M accepts x , ACCEPT

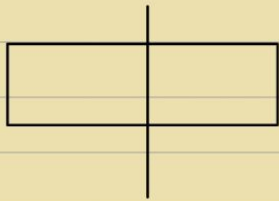
Else REJECT

if $E(M_1, M_2)$ accept $\Leftrightarrow$ M does not accept x



→ Divide and Conquer:

- Merge Sort



$$T(1) = 1 \rightarrow O(n)$$

$$T(n) = 2T(n/2) + O(n)$$

$$T(n) = O(n \log n)$$

- Given $A = [\quad]$
 $\rightarrow n$ elements
 $\& x \in A$

What is $\text{rank}(x)$ in A ?

$$O(n) \rightarrow \text{Easy}$$

- Order Statistics

Given $A = (\quad)$ and index i

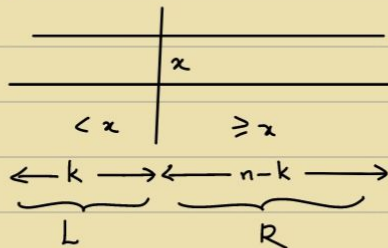
Find the i^{th} ranked element in A

$$O(n \log n) \rightarrow \text{Easy}$$

$$O(n) \rightarrow ?$$

$S(A, i)$

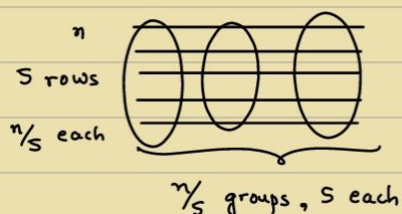
Pivot x



$$S(A, i) = \begin{cases} S(L, i), & \text{if } i \leq k \\ S(R, i-k), & \text{if } i > k \end{cases}$$

$$T(n) = O(n) + \max(T(k), T(n-k))$$

Choose a 'good' x



$n/5$ medians

Let x be median of these $n/5$ medians
 $S(n/5, n/10)$

$$T(n) = O(n) + T(n/5) + T(n/10)$$

$$T(n) = O(n)$$

$$T(n) = cn$$

$$\begin{aligned} \Rightarrow cn &= O(n) + c(n/5) + c(n/10) \\ &= O(n) + nc(1/5 + 1/10) \\ &< 1 \end{aligned}$$

$$= (0.1) cn = O(n)$$

$$n/3 + 2n/3 \rightarrow \text{Not } < 1$$

$\therefore$ Won't work

Optimal group size ?

14/8

→ Fast Fourier Transform (FFT) :

Input : Two polynomials of degree 'n-1' say $A(x)$ and $B(x)$

Output : Their product $C(x)$, i.e. $C(x) = A(x) \cdot B(x)$

$$A[n] \quad B[n] \longrightarrow C[2n-1]$$

↙
Coefficient Array

$$A(x) = \sum_{i=0}^{n-1} a_i x^i \quad \Bigg| \quad B(x) = \sum_{i=0}^{n-1} b_i x^i$$

$$C(x) = \sum_{i=0}^{2n-2} c_i x^i$$

↙
 c_i : 'Direct Formula'

↙
 $\sum_{j=0}^i a_j b_{i-j}$
 $O(n^2)$ algorithm

Can we do better ? i.e. $O(n \log n)$? FFT

Algorithm 'style'

- ① Select $(2n-1)$ or more points $x_1, x_2, x_3, \dots, x_n$ $[O(n)]$
- ② Evaluate $A(x_i)$ and $B(x_i)$ $[O(n^2)]$
- ③ Multiply $A(x_i) \cdot B(x_i) = C(x_i)$ $[O(n)]$
- ④ Interpolate $C(x_i)$'s to obtain $c(x)$ $[O(n^3)]$

Task: ② $\rightarrow O(n^2)$ to $O(n \log n)$

$$A(x) = A_e(x^2) + x A_o(x^2)$$

e.g. $A(x) = 5x^4 - 3x^3 + 7x^2 + 2x + 1$

$$A_e(x) = 5x^2 + 7x^2 + 1$$

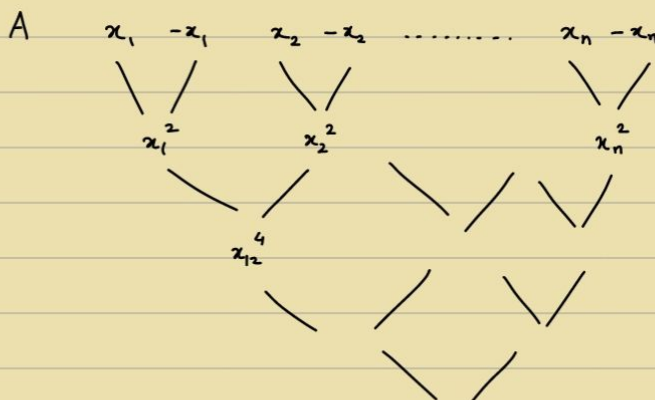
$$A_o(x) = -3x + 2$$

$$A(x_i) = A_e(x_i^2) + x_i \cdot A_o(x_i^2)$$

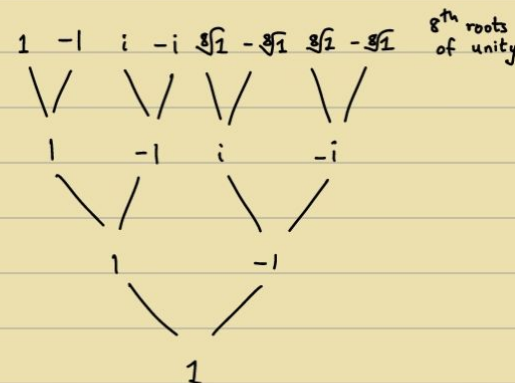
$$A(-x_i) = A_e(x_i^2) - x_i \cdot A_o(x_i^2)$$

$$T(n) = 2T(n/2) + O(n)$$

$$A_e(x_i^2) = A_{ee}(x_i^2) + x_i^2 \cdot A_{eo}(x_i^2)$$



n^{th} roots of unity



$$m \geq 2n-1, e^{i \frac{2k\pi}{n}}$$

Goal: To evaluate a polynomial 'A' on the n^{th} roots of unity

$$1, \omega, \omega^2, \dots, \omega^{n-1}$$

Input: $A[]$

Output: $(A[1], A[\omega], A[\omega^2], \dots, A[\omega^{n-1}])$

$$A = \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_m \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 & 1 & \dots \\ 1 & \omega & \omega^2 & \omega^4 & \dots \\ 1 & \omega^2 & \omega^4 & \dots & \dots \\ 1 & \omega^4 & \vdots & \dots & \dots \end{bmatrix}_{m \times m} \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_{m-1} \end{bmatrix} = \begin{bmatrix} A(1) \\ A(\omega) \\ A(\omega^2) \\ \vdots \\ A(\omega^{n-1}) \end{bmatrix}$$

Discrete Fourier Transform [Interpolation]

$$\text{FFT}(A, \omega):$$

if $(\omega = 1)$

```
return A(1);
```

else

$$A_e \leftarrow [A(0), A(\omega), \dots]$$
$$A_0 \leftarrow [A(1), A(2), \dots]$$
$$E[\cdot] \leftarrow \text{FFT}(A_e, \omega^2)$$
$$O[] \leftarrow \text{FFT}(A_0, \omega^2)$$

for $i = 0$ to $n-1$

$$A[i] \leftarrow E[i/2] + \omega^i \cdot O[i/2]$$
$$A(x) = A_e(x^2) + x \cdot A_0(x^2)$$
$$A(\omega^i) = E(\omega^{x_i}) + \omega^i \cdot O(\omega^{x_i})$$
$$A(-\omega^i) = E(\omega^{zi}) - \omega^i \cdot O(\omega^{zi})$$

for $i = 0$ to $n/2$

$$A[i] \leftarrow E[i] + \omega^i \cdot O[i]$$
$$A[i + m/2] \leftarrow E[i] + \omega^i \cdot O[i]$$
$$\therefore O(m \log n)$$

multiplication

Algorithm 'style'

- ① Select $m \geq (2n-1)$ or more points, namely n^{th} roots of unity

$$1, \omega, \omega^2, \omega^3, \dots, \omega^{m-1} \quad \left(\omega = e^{i \frac{2k\pi}{m}} \right) \quad (m: \text{exact power of } 2)$$

- ② Evaluate $A(x_i)$ and $B(x_i)$ $[O(n \cdot \log n)]$

$$\text{FFT}(A, \omega) \quad \& \quad \text{FFT}(B, \omega)$$

- ③ Multiply $A(x_i) \cdot B(x_i) = C(x_i)$ $[O(n)]$

- ④ Interpolate $c(x_i)$'s to obtain $c(x)$ $[O(n^3)]$

$$C(x) = A(x) \cdot B(x)$$

$$C(i) = A(i) \cdot B(i)$$

$$C(\omega) = A(\omega) \cdot B(\omega)$$

$$C(\omega^2) = A(\omega^2) \cdot B(\omega^2)$$

$$\begin{bmatrix} & \\ M & \end{bmatrix}_{m \times m} \cdot \begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_{m-1} \end{bmatrix} = \begin{bmatrix} c(1) \\ c(\omega) \\ c(\omega^2) \\ \vdots \\ c(\omega^{m-1}) \end{bmatrix}$$

$$M_{ij} = \omega_{ij}$$

$$\omega = e^{i \cdot \frac{2\pi}{n}}$$

$$M(w) = (i, j)^{th} \text{ entry, } w_{ij}$$

$$\begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_{m-1} \end{bmatrix} = M^{-1} \begin{bmatrix} c(1) \\ c(w) \\ \vdots \\ c(w^{m-1}) \end{bmatrix}$$

$$\begin{bmatrix} & & & & \\ & & & & \\ & & & & \\ & & & & \\ & & & & \end{bmatrix}_{m \times m} \begin{bmatrix} & & & & \\ & & & & \\ & & & & \\ & & & & \\ & & & & \end{bmatrix} = \begin{bmatrix} m & 0 & \dots & 0 \\ 0 & m & 0 & \dots & 0 \\ 0 & 0 & m & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \dots & \dots & \dots & 0 \end{bmatrix}$$

w_{ij} w^{-ij}

$$M^{-1} = \frac{1}{m} M(w^{-1})$$

$$\frac{1}{m} \text{FFT}(m w^{-1})$$

18/8, 21/8, 25/8

Absent - Not well

28/8

→ Greedy Strategy:

- Minimum Spanning Tree - Kruskal, Prims, etc
- Activity Selection
- Knapsack Problem
- Huffman coding
- Dijkstra's shortest path algorithm

→ Matroid Theory:

Def: A matroid is a pair $\langle S, I \rangle$, where

- (a) S is a finite non-empty set
- (b) I is a family of subsets of S . (Set of independent sets)

① Hereditary Property: If $A \subseteq S$ belongs to I (if $A \in I$),
Then, $\forall B \subseteq A$, $B \in I$

② Exchange Property : If $A \in \mathcal{I}$ and $B \in \mathcal{I}$, and $|B| > |A|$,

Then, $\exists x \in B/A$ s.t. $A \cup \{x\} \in \mathcal{I}$

Q) Given a matroid $\langle S, \mathcal{I} \rangle$ and given non-negative integer weight to element of S .
Find the maximum weight element of $\mathcal{I}$. (A subset of S)

$$w(A) = \sum_{x \in A} w(x) \quad \left| \quad \begin{array}{l} x \in S \\ w(x) \text{ is the weight of } x \end{array} \right.$$

• Universal Greedy Algo for weighted Matroids :

(1) Sort the elements of S in non-increasing order of weights.

(2) $A \leftarrow \emptyset$

(3) In that order, Let x be the root element

if $A \cup \{x\} \in \mathcal{I}$, then $A \leftarrow A \cup \{x\}$

• Proof of correctness / optimality :

Greedy-choice Property : The element $x \in S$ with the maximum weight belongs to some optimum solution.

Proof :

Let $A \in \mathcal{I}$ be an optimum solution

if $x \in A$, done

Suppose $x \notin A$, $\{x\} \in \mathcal{I}$

Let $A = \{a_1, a_2, \dots, a_k\}$

$\exists B = (A \cup \{x\}) - \{a_i\}$, $B \in \mathcal{I}$

$w(B) = w(x) + w(A) - w(a_i) \geq w(A)$

- Optimal sub-structure property :

M contracts to M'
 $\langle S, \mathcal{I} \rangle \quad \quad \langle S', \mathcal{I}' \rangle$

$A = A' \cup \{x\}$

$S' = S - \{x\}$

$\mathcal{I}' = \text{Sets of } \mathcal{I} \text{ with } x \text{ erased}$

$= \{B \mid B \cup \{x\} \in \mathcal{I}\}$

→ All sets in $\mathcal{I}$ that originally contained x

- Graph Matroid:

Given a edge-weighted graph $G = (V, E)$

Let $S_G = E$

$I_G = \{E' \subseteq E \mid E' \text{ does not contain a cycle}\}$

$M_G = \langle S_G, I_G \rangle$ is a matroid.

I_G is hereditary.

I_G has exchange property

$$|E_1| > |E_2|$$

→ Forest with $|V| - |E_2|$ Trees
→ Forest with exactly $|V| - |E_1|$ Trees

E_1 has an edge (u, v) s.t. u and v are in different trees in E_2 .

$$E_2 \in I$$

$$E_2 \cup \{(u, v)\} \in I_G$$

$\therefore$ Weighted Matroid

Quiz - 1

→ Knapsack Problem ✓

Greedy ✓

Activity Selection ✓

Kruskal's Algorithm ✓

Prim's Algorithm ✓

Cut Property ✓

Minimum Spanning Tree ✓

Strassen's Technique ✓ $O(n^{2.87})$

Binary Search ✓

Sorting - Counting Sort, Radix Sort

Merge Sort ✓

Karatsuba's Algorithm

Dijkstra's Algorithm ✓

Master's Theorem ✓

Fast Fourier Transform ✓

Equal_{TM}

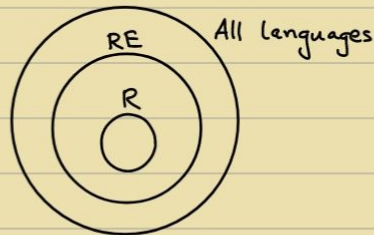
$A_{TM} = \{ \langle M, x \rangle, M \text{ accepts } x \}$

Halting_{TM}

Completeness Theory

Reduction

→ Recursive : Decidable $\leftarrow$ Turing Machine $\Rightarrow$ Accept if in L , Reject otherwise
Recursive Enumerable : Recognisable $\leftarrow$ TM $\Rightarrow$ Accept if in L , Halt/Loop otherwise
 $\rightarrow$ Partially / Semi - Decidable



Case - I : $\log_b a > k$

$$\therefore T(n) = \Theta(n^{\log_b a})$$

Case - II : $\log_b a = k$

(i) If $p > -1$:

$$\therefore T(n) = \Theta(n^k \log^{p+1} n)$$

(ii) If $p = -1$:

$$\therefore T(n) = \Theta(n^k \log(\log n))$$

(iii) If $p < -1$:

$$\therefore T(n) = \Theta(n^k)$$

Case - III : $\log_b a < k$

(i) If $p \geq 0$:

$$\therefore T(n) = \Theta(n^k \log^p n)$$

(ii) If $p < 0$:

$$\therefore T(n) = \Theta(n^k)$$

$$T(n) = aT(n/b) + f(n)$$

Conditions :

$$a \geq 1,$$

$$b > 1,$$

$$k \geq 0,$$

$$p \in \mathbb{R}$$

$$[\text{where } f(n) = \Theta(n^k \log^p n)]$$

Every B.S represents a TM.

only 01100100

A hand-drawn diagram illustrating a sequence of operations. It starts with a circle containing the binary string '01100100'. An arrow points from this circle to a rectangular box labeled 'Tm1'. Another arrow points away from the right side of the 'Tm1' box.

Handwritten notes on a whiteboard:

- Top left: "i that accepts Ld." (with "BS" written above it)
- Top center: "E 0 0p 10 11"
- Top right: "Order" (written vertically next to a bracketed list of numbers 0, 1, 2, 3, 4, 5)
- Center: A diagram showing a node labeled "Ld" with a circle around it, and the text "rm accepts" below it.
- Bottom left: "ation." and "→ rm accepts x"
- Bottom right: A red button with the text "WATCH NOW".

Handwritten notes on a piece of paper:

- Top left: "i that accepts Ld."
- Top center: "ES"
- Top right: "E 0 1 2 3 4 5 6 7 8 9 10 11"
- Right side: "Order" (written vertically)
- Center: A diagram showing a box labeled "Ld" with a circle labeled "Ld" inside it. Below the circle is "Ld" and "Ld".
- Bottom left: "ation." and "→ m accepts x"
- Bottom right: A red button with the text "WATCH NOW".

Handwritten notes on a piece of paper:

- Top left: "i that accepts Ld."
- Top center: "ES"
- Top right: "E 0 1 2 3 4 5 6 7 8 9 10 11"
- Right side: "Order" (written vertically)
- Center: A diagram showing a box labeled "Ld" with a circle labeled "Ld" inside it. Below the circle is "Ld" and "Ld".
- Bottom left: "ation." and "→ m accepts x"
- Bottom right: A red button with the text "WATCH NOW".

Handwritten notes on a whiteboard:

- Top left: "i that accepts Ld." (with "BS" written above it)
- Top right: "Order" written vertically next to a list of numbers 1, 2, 3, 4, 5.
- Center: A diagram showing a box labeled "Ld" with an arrow pointing to a circle labeled "Ld". Below the circle is the text "rm accepts".
- Bottom left: "x" and "rm accepts" written below the diagram.
- Bottom right: A red button with the text "ACCEPTED NOW".

Handwritten notes on a whiteboard:

- Top left: "i that accepts Ld." (with "BS" written above it)
- Top right: "Order" written vertically next to a list of numbers 1, 2, 3, 4, 5.
- Center: A diagram showing a box labeled "Ld" with an arrow pointing to a circle labeled "Ld". Below the circle is the text "rm accepts".
- Bottom left: "x" and "rm accepts" written below the diagram.
- Bottom right: A red button with the text "ACCEPTING NOW".

Handwritten notes on a whiteboard:

- Top left: "i that accepts Ld." (with "BS" written above it)
- Top center: "E 0 0p 10 11"
- Top right: "Order" (written vertically next to a bracketed list of numbers 0 through 5)
- Center: A diagram showing a node labeled "Ld" with a circle around it, and the text "rm accepts" below it.
- Bottom left: "x" and "rm accepts" (with an arrow pointing to the "rm accepts" text)
- Bottom right: A red button that says "WATCH NOW".

Diagram illustrating the construction of a secure commitment scheme using a one-way function F and a decommitment key D .

Commitment Phase:

- Input m is hashed by H to produce $x = H(m)$.
- x is combined with a random string r and passed through the one-way function F to produce the commitment $c = F(x \| r)$.
- The commitment c is sent to the receiver.

Decommitment Phase:

- The sender provides the commitment c and the decommitment key D to the receiver.
- The receiver uses D to extract x from c and then recovers m by computing $H^{-1}(x)$.

Security Properties:

- Correctness:** The receiver can always recover the original message m using the decommitment key D .
- Binding:** The sender cannot decommit to a different message $m' \neq m$ because finding r' such that $F(x' \| r') = c$ is computationally infeasible due to the one-way property of F .
- Concealment:** The commitment c does not reveal any information about m because F is a one-way function.

$Halt = \{ \langle M, w \rangle \mid M \text{ halts on } w \}$
 Halt is not recursive & undecidable.

Proof: - let us assume halt is R.
 $\Rightarrow \exists$ a TM (decider) that accepts halt and rejects all other inputs (never goes into loop).
 $\Rightarrow$ let us name that decider as D.
 $L(D) = \text{halt}$.

Yes if M halts on w.
 No if M does not halt on w.

Yes is a contradiction.

$\Rightarrow$ there is also contradiction to our assumption is wrong.
 $\Rightarrow$ Halt is not R, undecidable.

Strassen's Matrix Multiplication

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \times \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} \xrightarrow{C+T \rightarrow} \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} \xrightarrow{\downarrow R} \begin{bmatrix} B_{11} & A_{11} & A_{22} & B_{22} \end{bmatrix}$$

$$P = (A_{11} + A_{22})(B_{11} + B_{22})$$

$$Q = B_{11}(A_{21} + A_{22})$$

$$R = A_{11}(B_{12} - B_{22})$$

$$S = A_{22}(B_{21} - B_{11})$$

$$T = B_{22}(A_{11} + A_{12})$$

$$U = (A_{21} - A_{11})(B_{11} + B_{12})$$

$$V = (B_{21} + B_{22})(A_{12} - A_{22})$$

$$C_{11} = P + S - T + V$$

$$C_{12} = \overline{R} + T \quad \textcircled{RA \oplus}$$

$$C_{21} = \overline{Q} + S$$

$$C_{22} = P + R - Q + U$$

No. of multiplies = 7
 " adds = 18

$$T(n) = \begin{cases} 1 & \text{if } n \leq 2 \\ 7T(n/2) + 18 & \text{if } n > 2 \end{cases}$$

$$a = 7$$

$$b = 2$$

$$f(n) = n^2 \log_2 7 = (\log_2 7) \cdot 2 \log_2 n$$

$$O(n^{2.81})$$

Karatsuba's Multiplication Algorithm

$X = 7768$ $a = 77$ $b = 68$
 $Y = 9276$ $c = 92$ $d = 76$

Step 1. Compute $a * c = 77 * 92 = 7084$ → 70840000
Step 2. Compute $b * d = 68 * 76 = 5168$ → 5168
Step 3. Compute $(a+b) * (c+d)$
 $(77+68) * (92+76)$
 $(145) * (168) = 24360$
Step 4. Compute $24360 - 7084 - 5168 = 12108$

$7768 * 9276 = 72055968$


```
def FFT(P) :
```

```
# P - [p0, p1, ..., pn-1] coeff representation
```

```
n = len(P) # n is a power of 2
```

```
if n == 1:
```

```
    return P
```

```
 $\omega = e^{\frac{2\pi i}{n}}$ 
```

```
 $P_e, P_o = [p_0, p_2, \dots, p_{n-2}], [p_1, p_3, \dots, p_{n-1}]$ 
```

```
 $y_e, y_o = \text{FFT}(P_e), \text{FFT}(P_o)$ 
```

```
y = [0] * n
```

```
for j in range(n/2):
```

```
     $y[j] = y_e[j] + \omega^j y_o[j]$ 
```

```
     $y[j + n/2] = y_e[j] - \omega^j y_o[j]$ 
```

```
return y
```

FFT $P(x) : [p_0, p_1, \dots, p_{n-1}]$
 $\omega = e^{\frac{2\pi i}{n}} : [\omega^0, \omega^1, \dots, \omega^{n-1}]$

$n = 1 \Rightarrow P(1)$

FFT $P_e(x^2) : [p_0, p_2, \dots, p_{n-2}]$
 $[\omega^0, \omega^2, \dots, \omega^{n-2}]$

$y_e = [P_e(\omega^0), P_e(\omega^2), \dots, P_e(\omega^{n-2})]$

FFT $P_o(x^2) : [p_1, p_3, \dots, p_{n-1}]$
 $[\omega^0, \omega^2, \dots, \omega^{n-2}]$

$y_o = [P_o(\omega^0), P_o(\omega^2), \dots, P_o(\omega^{n-2})]$

$P(\omega^j) = y_e[j] + \omega^j y_o[j]$

$P(\omega^{j+n/2}) = y_e[j] - \omega^j y_o[j]$

$j \in \{0, 1, \dots, (n/2 - 1)\}$

$y = [P(\omega^0), P(\omega^1), \dots, P(\omega^{n-1})]$

IFFT(<values>) $\Leftrightarrow$ FFT(<values>) with $\omega = \frac{1}{n} e^{\frac{-2\pi i}{n}}$

```
def FFT(P) :
```

```
# P - [p0, p1, ..., pn-1] coeff rep
```

```
n = len(P) # n is a power of 2
```

```
if n == 1:
```

```
    return P
```

```
 $\omega = e^{\frac{2\pi i}{n}}$ 
```

```
 $P_e, P_o = P[:2], P[1:2]$ 
```

```
 $y_e, y_o = \text{FFT}(P_e), \text{FFT}(P_o)$ 
```

```
y = [0] * n
```

```
for j in range(n/2):
```

```
     $y[j] = y_e[j] + \omega^j y_o[j]$ 
```

```
     $y[j + n/2] = y_e[j] - \omega^j y_o[j]$ 
```

```
return y
```

```
def IFFT(P) :
```

```
# P - [P(ω0), P(ω1), ..., P(ωn-1)] value rep
```

```
n = len(P) # n is a power of 2
```

```
if n == 1:
```

```
    return P
```

```
 $\omega = (1/n) * e^{\frac{-2\pi i}{n}}$ 
```

```
 $P_e, P_o = P[:2], P[1:2]$ 
```

```
 $y_e, y_o = \text{IFFT}(P_e), \text{IFFT}(P_o)$ 
```

```
y = [0] * n
```

```
for j in range(n/2):
```

```
     $y[j] = y_e[j] + \omega^j y_o[j]$ 
```

```
     $y[j + n/2] = y_e[j] - \omega^j y_o[j]$ 
```

```
return y
```

Harith Gessagolam

→ Dynamic Programming :

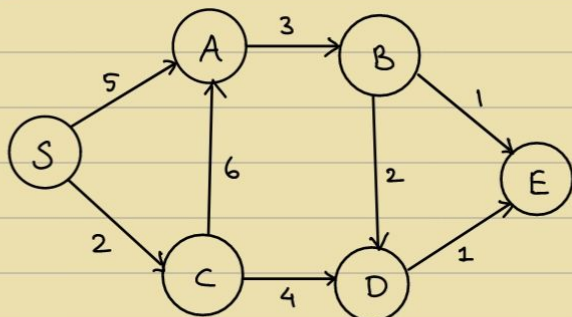
- A bag of subproblems S.T. it forms a DAG of 'smallest' to the 'largest' in Topological Sorted order.
- Problem : Shortest Path in DAGs

Input : Weighted Directed Acyclic Graph

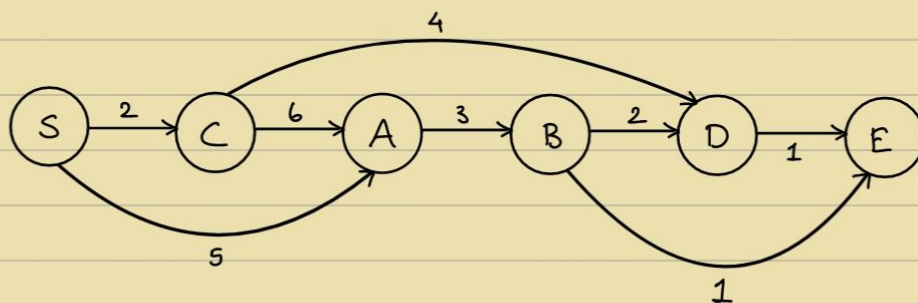
Source S and Destination D

Output : Shortest path from S and D.

Example :



Topological sort :



$$P(S, D) = \min \begin{cases} 2 + P(S, B) \\ 4 + P(S, C) \end{cases}$$

Termination condition : $P(S, S) = 0$

0	2	5	8	6	X
$S \rightarrow S$	$S \rightarrow C$	$S \rightarrow A$	$S \rightarrow B$	$S \rightarrow D$	$S \rightarrow E$

$$P(S, i) = \min_{(u, i) \in E} (P(S, u) + D(u, i))$$

↗ in-neighbours

$$S \rightarrow A : \min(5, 8)$$

$$S \rightarrow D : \min(8, 10)$$

- Longest path in DAG:

$$P(s, i) = \max_{(u, i) \in E} (P(s, u) + D(u, i))$$

Topological sorting: $O(V+E)$

0	2	8	11	13	14
---	---	---	----	----	----

$S \rightarrow S \quad S \rightarrow C \quad S \rightarrow A \quad S \rightarrow B \quad S \rightarrow D \quad S \rightarrow E$

If efficient algorithm for finding longest path for general graphs,
then $P = NP$

- Longest Increasing Subsequence:

Input: Array of numbers

Output: LIS

Every substring is a subsequence, but not vice-versa. ↗ may not be continuous

e.g. $a_1, a_2, a_3, \dots, a_n$

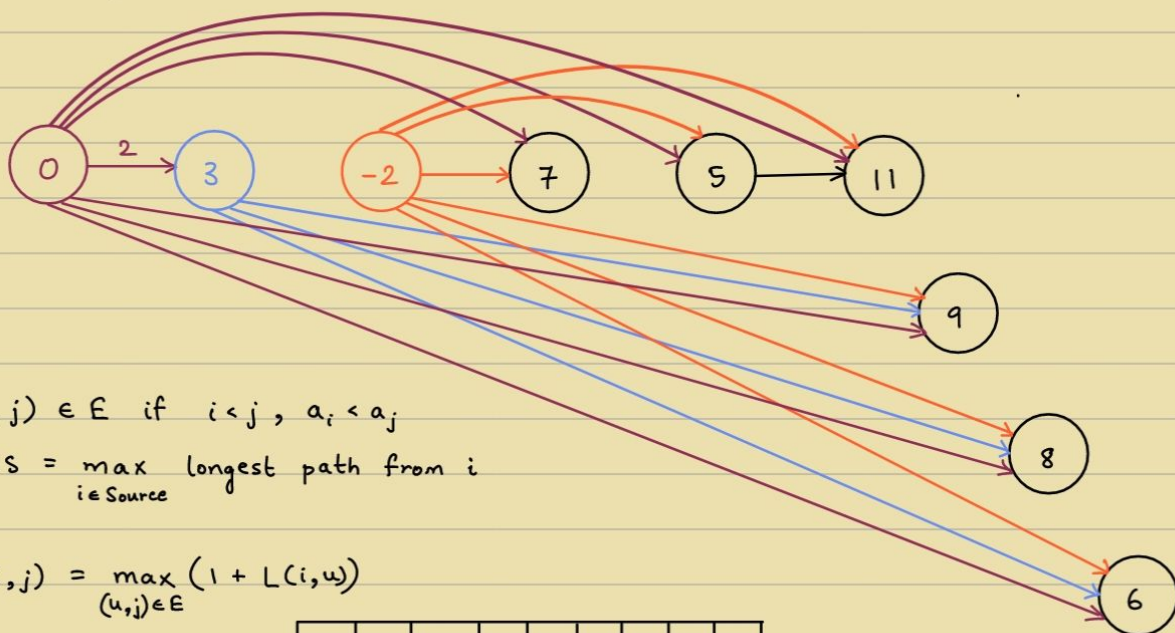
$a_{i_1} < a_{i_2} < a_{i_3} < \dots < a_{i_n}$
where $i_1 < i_2 < i_3 < \dots < i_n$ } → Sequence

e.g. 0, 3, -2, 7, 5, 11, 9, 8, 6

Sol: 0, 3, 7, 11 → length: 4

Viterbi's Algo, CYK Algo: Dynamic Programming

↓
Signal Processing ToC



$(i, j) \in E$ if $i < j, a_i < a_j$

$LIS = \max_{i \in \text{Source}} \text{longest path from } i$

$$L(i, j) = \max_{(u, j) \in E} (1 + L(i, u))$$

0	1	$-\infty$	2	2	3	3	3	3
0	3	-2	7	5	11	9	8	6

⇒ 0, 3, 7, 11

- Edit Distance: (Levenshtein distance)

$O(|E| \cdot k)$

$O(n^2) \rightarrow O(n \log n) ?$

Worst case scenario

Operations: Insert, Delete, Overwrite

e.g. EARTH $\rightarrow$ HEART

① Insert H
HEARTH

② Delete H
HEART

e.g. SUNNY $\rightarrow$ SNOWY

① Overwrite U $\rightarrow$ N

② Overwrite N $\rightarrow$ O

③ Overwrite N $\rightarrow$ W

WAP that input 2 strings and finds the edit distance b/w them.

11/9

Review:

Dynamic Programming

e.g. ① Shortest / Longest Path in DAG

② Longest Increasing Subsequence

- Common Traits of D.P:

- Bag / DAG of Subproblems

- Subproblems are usually some part of the Input.

- Recursive relation

- Edit Distance:

$x = x_1 x_2 x_3 \dots x_n$

$y = y_1 y_2 y_3 \dots y_m$

Add as many blanks to x and y S.T. they are aligned.

Cost of Alignment: No. of mismatches

EARTH $\Rightarrow$ _EARTH
HEART HEART_

Subproblems :

(prefix) 1st i characters of x

(prefix) 1st j characters of y

$$E(i, j) \rightarrow \text{Goal} : E(n, m)$$

$$\left. \begin{array}{c} x_1 x_2 \dots x_i \\ y_1 y_2 \dots y_j \end{array} \right\} \text{Three cases : (a) } \left. \begin{array}{c} x_i \\ - \end{array} \right\} \Rightarrow E(i, j) = E(i-1, j) + 1$$

$$(b) \left. \begin{array}{c} - \\ y_j \end{array} \right\} \Rightarrow E(i, j) = E(i, j-1) + 1$$

$$(c) \left. \begin{array}{c} x_i \\ y_j \end{array} \right\} \Rightarrow E(i, j) = \begin{cases} E(i-1, j-1) & x_i = y_j \\ E(i-1, j-1) + 1 & x_i \neq y_j \end{cases}$$

$$\therefore E(i, j) = \min \left(\begin{array}{c} 1 + E(i-1, j) \\ 1 + E(i, j-1) \\ \text{diff}(x_i, y_j) + E(i-1, j-1) \end{array} \right)$$

$$\text{diff}(x_i, y_j) = \begin{cases} 0, & x_i = y_j \\ 1, & x_i \neq y_j \end{cases}$$

Termination condition :

$$E(0, j) = j$$

$$E(i, 0) = i$$

		x_1	x_2	x_3		x_n
	0	1	2	3		n
y_1	1					
y_2	2					
y_3	3					
$\vdots$	$\vdots$					
y_m	m					$E(n,m)$

e.g.

		H	E	A	R	T
	0	1	2	3	4	5
E	1	1	1	2	3	4
A	2	2	2	1	2	3
R	3	3	3	2	1	2
T	4	4	4	3	2	1
H	5	4	5	4	3	2

Fill it row-wise

to not get stuck anywhere.

e.g. EXPONENTIAL
POLYNOMIAL

		E	X	P	O	N	E	N	T	I	A	L
	0	1	2	3	4	5	6	7	8	9	10	11
P	1	1	1	2	3	4	5	6	7	8	9	10
O	2	2	2	3	2	3	4	5	6	7	8	9
L	3	3	3	3	3	3	4	5	6	7	8	9
Y	4	4	4	4	4	4	4	5	6	7	8	9
N	5	5	5	5	5	4	5	4	5	6	7	8
O	6	6	6	6	5	5	5	5	5	6	7	8
M	7	7	7	7	6	6	6	6	<u>6</u>	6	7	8
I	8	8	8	8	7	7	7	7	7	6	7	8
A	9	9	9	9	9	8	8	8	8	7	8	9
L	10	10	10	10	9	9	9	9	9	8	7	<u>6</u>

Note : IAL is not required $\because$ both strings end with 'IAL'

Answer : 6

Working :

$dp[m+1][n+1]$

where $dp[i][j]$: min. edits to convert first i chars of EXPONENTIAL to first j chars of POLYNOMIAL

$dp[0][j] = j$

$dp[i][0] = i$

for $i > 0, j > 0$:

if $s_1[i-1] == s_2[j-1]$:

$dp[i][j] = dp[i-1][j-1]$

else :

$dp[i][j] = 1 + \min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])$

$\swarrow$ Delete $\swarrow$ Insert $\swarrow$ Replace

Result : $dp[i][10] = \underline{\underline{6}}$

15/9

→ Review:

Dynamic Programming

Ex 1: Shortest / Longest Path in DAG

Ex 2: Longest Increasing Subsequence

Ex 3: Edit distance

Ex. 4: Chain Matrix Multiplication

→ General Technique:

- Creating subproblems using either prefix or some part of input.
- Relate the subproblems from easier to less easy in topological sorted order.

Input: Chain of Matrices

$$M_{P_0 \times P_1}^{(1)}, M_{P_1 \times P_2}^{(2)}, M_{P_2 \times P_3}^{(3)}, \dots, M_{P_{n-1} \times P_n}^{(n)}$$

$$\text{Output: } M_{P_0 \times P_n} = \prod_{i=1}^n M_{P_{i-1} \times P_i}$$

Cost: no. of elementary multiplications

Note: Notation:

$$M_{p,q} \times M_{q,r} \rightarrow pqr \text{ multiplications}$$

$$\text{Example: } M_{100 \times 10}^{(1)} \times M_{10 \times 1000}^{(2)} \times M_{1000 \times 1}^{(3)} \times M_{1 \times 100}^{(4)}$$

$$\text{Total multiplications: } 10^6 + 10^5 + 10^7 = 11100000$$

$$\left((M^{(1)} \times M^{(2)}) \times (M^{(3)} \times M^{(4)}) \right)$$

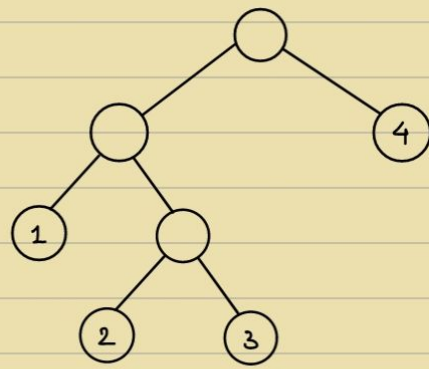
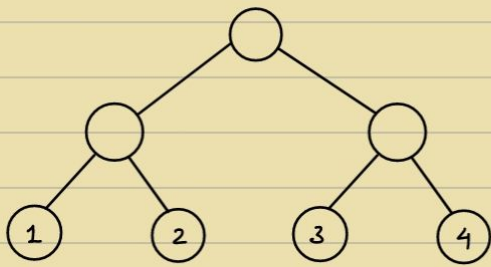
$$\text{But, } \left[M^{(1)} \times (M^{(2)} \times M^{(3)}) \right] \times M^{(4)}$$

$$10^3 + 10^4 + 10^4 = 21000$$

Q) How many different ways are there to parenthesize a chain of n matrices?

$$\frac{1}{n+1} \binom{2n}{n}$$

[Catalan numbers]



$C(i, j)$

Goal : To find $C(1, n)$

$$C(i, i+1) = P_{i-1} P_i P_{i+1} \leftarrow \text{Easy ones}$$

$$\begin{bmatrix} M^{(i)} \\ P_{i-1} \times P_i \end{bmatrix}$$

Now,

Only P_i 's are given

$$C(i, j) = C(i, k) + C(k+1, j) + P_i P_j P_k$$

(if last step was given)

$$\text{Suppose } \begin{pmatrix} M^{(i)} \times M^{(k)} \\ M^{(k+1)} \times M^{(j)} \end{pmatrix}$$

Termination condition : $C(i, i) = 0$

$$C(i, j) = \min_{i \leq k < j} \{ C(i, k) + C(k+1, j) + P_{i-1} P_j P_k \}$$

	1	2	3	4
1	0	10^6	11000	21000
2		0	10^4	12000
3			0	10^5
4				0

Goal (points to 21000)

Not necessary (points to the bottom row)

$$C(1, 3) = \min \begin{cases} C(1, 2) + C(3, 3) + P_0 P_2 P_3 \\ C(1, 1) + C(2, 3) + P_0 P_1 P_3 \end{cases}$$

$$C(1, 3) = \min \begin{cases} C(1, 3) + C(3, 4) + P_1 P_3 P_4 \\ C(2, 2) + C(2, 4) + P_1 P_2 P_4 \end{cases} = \min \begin{cases} 11000 + 1000 \\ 10^5 + \dots \end{cases}$$

$$C(1, 3) = \min \begin{cases} C(1, 1) + C(2, 4) + P_0 P_1 P_4 \\ C(1, 2) + C(3, 4) + P_0 P_2 P_4 \\ C(1, 3) + C(4, 4) + P_0 P_3 P_4 \end{cases}$$

18/9

→ Review :

D.P :

- ① Paths in DAG
- ② LIS
- ③ Edit distance
- ④ Chain Matrix Multiplication

→ Ex. Shortest Reliable Paths :

Input : Graph (Edge weighted), s (start), t (end), integer k

Output : The shortest path from s to t in G using $\leq k$ edges

• Subproblems :

$$\forall x \in V, \forall i < k$$

Let $\text{dist}(u, i)$ be the length of the shortest path from s to u , using i hops

• Main problem :

$$\min_{i \leq k} \text{dist}(t, i)$$

Recurrence Relation :

$$\text{dist}(u, i) = \min_{(v, u) \in E} (\text{dist}(v, i-1) + w(v, u))$$

Easiest Subproblem :

$$\text{dist}(s, 0) = 0 \quad (\text{Termination condition})$$

$$\text{dist}(s, > 0) = \infty \quad (\text{OR}) \quad \text{dist}(u, 0) = \infty \quad \begin{matrix} \downarrow \\ \neq s \end{matrix}$$

→ Ex. All Pair shortest Path : $O(n^3)$

Input : G

Output : $\forall u, v$, Length of shortest path from u to v .

• Subproblems :

Let $\text{dist}(u, v, k)$ be the length of the shortest path from u to v , using minimalistic nodes among $1, 2, \dots, k$

• Main Problem :

$$k = |V|$$

Recurrence relation :

$$\text{dist}(u, v, k) = \min(\text{dist}(u, v, k-1), \text{dist}(u, k, k-1) + \text{dist}(k, v, k-1))$$



$$\text{dist}(u, v, 0) = \begin{cases} w(u, v), & \text{if } (u, v) \in E \\ \infty, & \text{otherwise} \end{cases}$$

Normal $\rightarrow O(|E| \cdot \log(|V|))$

With fibonacci heap $\rightarrow O(|E| + |V| \log(|V|))$

$\rightarrow$ Review for midsem:

① Dynamic Programming

- Shortest / longest Path in DAG
- Longest Increasing Subsequence
- Edit Distance
- All Pair shortest Path
- Chain Matrix Multiplication
- Shortest Reliable Path
- Knapsack
- Independent set in Tree
- Polygon Triangulation
- Other questions in Prob. set

② Greedy Algorithm

- Huffman codes
- Dijkstra's Algo
- Knapsack (0-1 & Fractional)
- Activity Selection
- Minimum / Maximum Spanning Tree (Prims Algo, Krushkal's Algo)
- Matroid Theorem
- Other questions in Prob. set

③ Divide and Conquer

- Merge Sort
- Integer Multiplication
- Matrix Multiplication
- Polynomial Multiplication
- Other questions in Prob. set

④ Computability Theory

- Church-Turing Thesis
- Model of Computation (TM)
- Diagonalization
- (a) Recognizability
- (b) Decidability
- Mapping Reduction

• Binary Search

$T(n)$ ——— Algorithm RBinSearch(l, h, key)

```

{
  if ( $l == h$ )
  {
    if ( $A[l] == key$ )
      return  $l$ ;
    else return  $0$ ;
  }
  else
  {
     $mid = (l + h) / 2$ ;
    if ( $key == A[mid]$ )
      return  $mid$ ;
    if ( $key < A[mid]$ )
      return RBinSearch( $l, mid - 1, key$ );
    else
      return RBinSearch( $mid + 1, h, key$ );
  }
}

```

$T(n/2)$ ———

$$T(n) = \begin{cases} 1, & n = 1 \\ T(n/2), & n \geq 1 \end{cases}$$

• Merge Sort :

Algorithm MergeSort(l, h)

```

{
  if ( $l < h$ )
  {
     $mid = (l + h) / 2$ ;
    MergeSort( $l, mid$ );
    MergeSort( $mid + 1, h$ );
    Merge( $l, mid, h$ );
  }
}

```

Algorithm Merge(A, B, m, n)

```

{
   $i = 1; j = 1; k = 1$ ;
  while ( $i \leq m$  &&  $j \leq n$ )
  {
    if ( $A[i] < B[j]$ )
       $C[k++] = A[i++]$ ;
    else
       $C[k++] = B[j++]$ ;
  }
  for ( $i \leq m; i++$ )
     $C[k++] = A[i]$ ;
  for ( $j \leq n; j++$ )
     $C[k++] = B[j]$ ;
}

```

$$T(n) = \begin{cases} 1, & n = 1 \\ 2T(n/2) + n, & n > 1 \end{cases}$$

• MST :

no. of MSTs = $|E| C_{|V|-1} - \text{no. of cycles}$

Prims $\rightarrow$ Initially, take the min. cost edge
(Tree) Then, take the connected min. cost edge.

Kruskals $\rightarrow$ Always take the min. cost edge. $\rightarrow O(|V| \cdot |E|)$
(7 Cycle) Don't consider loop edges. Min heap $\rightarrow O(|V| \cdot \log |V|)$

Dijkstra's $\rightarrow O(|V| \cdot |V|)$

- Dynamic Programming:

- Multi-Stage Graph: (Shortest Path in DAG)

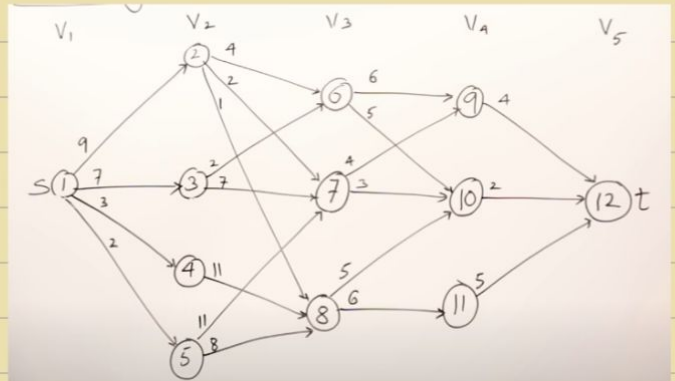
stage vertex no

$$Cost(i, j) = \min_{\substack{4, 12 \in E \\ l \in V_{i+1}}} \{c(j, l) + Cost(i+1, l)\}$$

V	1	2	3	4	5	6	7	8	9	10	11	12
Cost	16	7	9	18	15	7	5	7	4	2	5	0
d	2/3	7	6	8	8	10	10	10	12	12	12	12

for longest Path $\rightarrow$ Multiply by -1
costs

& Multiply result by -1 to
get correct cost.



- All pair shortest Path: (Floyd-Warshall)

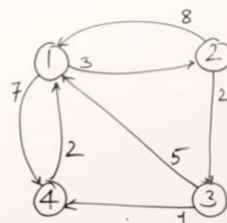
All Pairs Shortest Path

$$A^1 = \begin{bmatrix} 1 & 0 & 2 & 3 & 4 \\ 2 & 8 & 0 & 2 & 15 \\ 3 & 5 & 8 & 0 & 1 \\ 4 & 2 & 5 & \infty & 0 \end{bmatrix}$$

$$A^2 = \begin{bmatrix} 1 & 0 & 2 & 3 & 4 \\ 2 & 8 & 0 & 2 & 15 \\ 3 & 5 & 8 & 0 & 1 \\ 4 & 2 & 5 & 7 & 0 \end{bmatrix}$$

$$A^3 = \begin{bmatrix} 1 & 0 & 2 & 3 & 4 \\ 2 & 7 & 0 & 2 & 3 \\ 3 & 5 & 8 & 0 & 1 \\ 4 & 2 & 5 & 7 & 0 \end{bmatrix}$$

$$A^4 = \begin{bmatrix} 1 & 0 & 2 & 3 & 4 \\ 2 & 5 & 0 & 2 & 3 \\ 3 & 3 & 6 & 0 & 1 \\ 4 & 2 & 5 & 7 & 0 \end{bmatrix}$$



$$A^0 = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 1 & 0 & 3 & \infty & 7 \\ 2 & 8 & 0 & 2 & \infty \\ 3 & 5 & \infty & 0 & 1 \\ 4 & 2 & \infty & \infty & 0 \end{bmatrix}$$

$$A[i, j] = \min \left\{ \underbrace{A[i, j]}^{k-1}, \underbrace{A[i, k] + A[k, j]}^{k-1} \right\}$$

for (k=1; k<=n; k++)

for (i=1; i<=n; i++)

for (j=1; j<=n; j++)

$$A[i, j] = \min(A[i, j], A[i, k] + A[k, j])$$

A⁰
2
3
4

- Matrix chain multiplication:

Matrix Chain Multiplication

$$A_1 \cdot A_2 \cdot A_3 \cdot A_4$$

$$\begin{array}{ccccc} (5 \times 4) & (4 \times 6) & (6 \times 2) & (2 \times 7) & \\ d_0 & d_1 & d_2 & d_3 & d_4 \end{array}$$

$$4] = \min \left\{ \begin{array}{l} m[1,1] + m[2,4] + 5 \times 4 \times 7, \quad m[1,2] + m[3,4] + 5 \times 6 \times 7 \\ 0 \quad 104 + 140 \quad \quad \quad 120 + 84 + 210 \\ m[1,3] + m[4,4] + 5 \times 2 \times 7 \\ 88 + 0 + 70 \end{array} \right\}$$

$$m[i, j] = \min \left\{ m[i, k] + m[k+1, j] + d_{i-1} * d_k * d_j \right\}$$

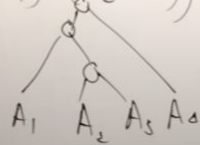
m	1	2	3	4
1	○	120	88	158
2		○	48	104
3			○	84
4				○

$$O(n^3)$$

	1	2	3	4
1		1	1	3
2			2	3
3				3
4				

$$(A_1 \cdot A_2 \cdot A_3) (A_4)$$

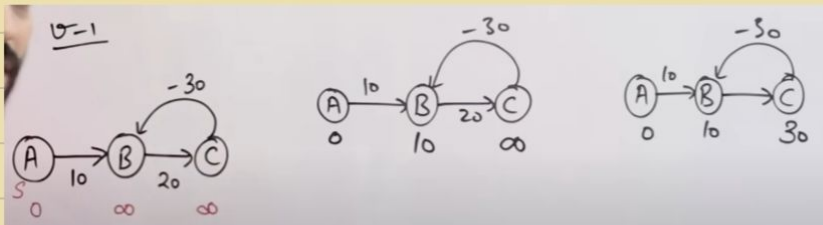
$$((A_1) \cdot (A_2 \cdot A_3)) \cdot (A_4)$$

~~| | | | |
|---|--|---|----|
| 3 | | 0 | 84 |
| 4 | | 0 | |~~

S	1	2	3	4
1		1	1	3
2			2	3
3				3
4				

- Bellman Ford:

Relax ' $|v| - 1$ ' times



- 0/1 Knapsack:

$$m=8 \quad P=\{1, 2, 5, 6\}$$

$$n=4 \quad \omega = \{2, 3, 4, 5\}$$

	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
1	0	0	1	1	1	1	1	1	1
2	0	0	1	2	2	3	3	3	3
3	0	0	1	2	5	5	6	7	7
4	0	0	1	2	5	6	6	7	8

$$V[i, \omega] = \max \{V[i-1, \omega], V[i-1, \omega - w[i]] + P[i]\}$$

$$V[4,8] = \max \left\{ V[3,8], V[3,8-5] + 6 \right\}$$

7, 2+6

$$\begin{array}{cccc} x_1 & x_2 & x_3 & x_4 \\ \left\{ \begin{array}{cccc} 0 & 1 & 0 & 1 \end{array} \right\} & 8-6=2 & & \\ & 2-2=0 & & \end{array}$$

- Travelling Salesman Problem:

$$g(i, S) = \min_{k \in S} \{ C_{ik} + g(k, S - \{i\}) \}$$

$$g(1, \{2, 3, 4\}) = \min_{k \in \{2, 3, 4\}} \{ C_{1k} + g(k, \{2, 3, 4\} - \{1\}) \}$$

Diagram showing a tree structure for the TSP problem starting from node 1, branching to nodes 2, 3, and 4.

Costs for each branch:

- Node 2: $C_{12} + g(2, \{3, 4\})$
- Node 3: $C_{13} + g(3, \{2, 4\})$
- Node 4: $C_{14} + g(4, \{2, 3\})$

Distance matrix:

	1	2	3	4
1	0	10	15	20
2	5	0	9	10
3	6	13	0	12
4	8	8	9	0

Diagram showing a complete graph with 4 nodes and all possible edges.

- Longest Common Subsequence:

Sequence A: a b d

Sequence B: a b c d

DP Table:

		a	b	c	d
0	0	0	0	0	0
b	1	0	0	1	1
d	2	0	0	1	2

Arrows indicate the path for the longest common subsequence: (0,0) → (1,2) → (2,4).

Recurrence Relation:

$$\text{if } (A[i] = B[j])$$

$$\text{LCS}[i, j] = 1 + \text{LCS}[i-1, j-1]$$

$$\text{else}$$

$$\text{LCS}[i, j] = \max(\text{LCS}[i-1, j], \text{LCS}[i, j-1])$$

Complexity: $O(m \times n)$

- Longest Increasing Subsequence:

Sequence: 3 4 -1 0 6 2 3

DP Table:

1	1	1	1	1	1	1
---	---	---	---	---	---	---



Sequence: 3 4 -1 0 6 2 3

DP Table:

1	2	1	2	3	3	4
---	---	---	---	---	---	---

Recurrence Relation:

$$j: 0 \rightarrow i-1$$

$$\text{if } \text{arr}[j] < \text{arr}[i]$$

$$\text{LIS}[i] = \max(\text{LIS}[i], \text{LIS}[j] + 1)$$

29/7

→ Algorithms and Complexity :

• A few wonders :

- Follow your passion.

→ Graph Algorithms

→ Number Theoretic Algorithms

→ String Theoretic Algorithms

⋮

→ Computational Physics

→ Computational Chemistry

→ Computational Biology

→ Computational Mathematics

→ Computational Economics

→ Computational Music

→ Computational Finance

• "Interesting" Problems :

- Tractability ?

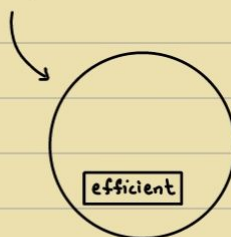
(Feasible)

- Efficiency ?

($\equiv$ Polynomial Time Algo)

↳ Convention

$O(2^n) \rightarrow$ Restating the problem



Closed under Addition, Subtraction & Multiplication.

$\therefore$ Using an algorithm as a subroutine

Intuitively, Polynomial Time works.

Algo - runtime : No. of steps taken by the Turing machine

TM : Model of Computation

TM : Model of Complexity

T.M may not be able to simulate $O(2^n)$ efficiently.

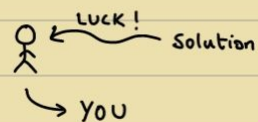
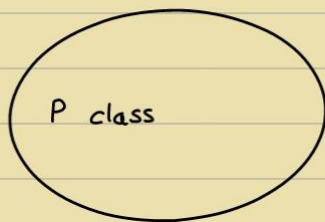
Worst case - Asymptotically - Polynomial : Efficient

• $TIME(n) = \{ L \mid L \text{ is decided by some DTM 'M', in time } O(n) \}$

$$P = \bigcup_{k=0}^{\infty} TIME(n^k)$$

↳ Polynomial class $\rightarrow$ Interesting

i.e. Feasible and Efficient solution can exist.



$x \in L$: Proof / Witness certificate

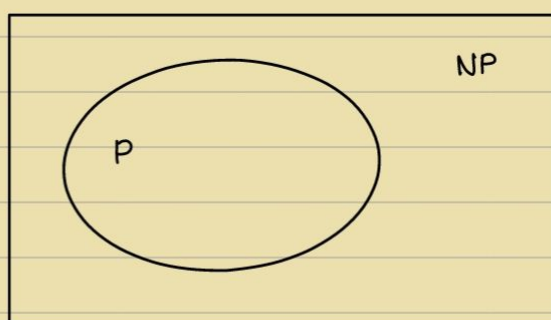
• $CLIQUE = \{ \langle G, k \rangle \mid G \text{ has a clique of size } k \}$

- Trial and Error strategy (Keep trying)

No. of trials $\uparrow$

Luck can play a role, i.e. when solution is given - checking is in P

$\therefore$ Interesting $\rightarrow$ NP



$Clique \in NP$

Given the solution, can you recognise/decide whether the solution is correct or not? i.e. How difficult it is to check.

If this problem is in P $\Rightarrow$ main problem in P

i.e. If problem $\rightarrow$ Uninteresting

But if we find that verifying solution for that problem $\rightarrow$ Interesting

Then, Problem $\rightarrow$ Interesting

If problem is beyond NP $\Rightarrow$ even less interesting but can be turned into even more interesting.

i.e. even if solution is given, difficult to verify solution.

e.g. Given $n \times n$ chessboard & Given winning strategy for winning every chess game in that board, as $n \rightarrow \infty \Rightarrow$ Outside NP

Intuition:

• Needle & Haystack

If you're finding needle in the haystack:

If you know what a needle is $\Rightarrow$ lift it and verify: NP

If you can't differentiate needle from haystack in Polynomial time
 $\Rightarrow$ You don't know you picked needle or not: Beyond NP

If Interaction with alien / Randomness (Alien has proof): IP-Space
 $\hookrightarrow$ can be verified

Complexity Theory $\rightarrow$ Hard (P, NP)

Beyond Complexity Theory,

Computability Theory $\rightarrow$ Really Hard (R, RE)

Open Problems:

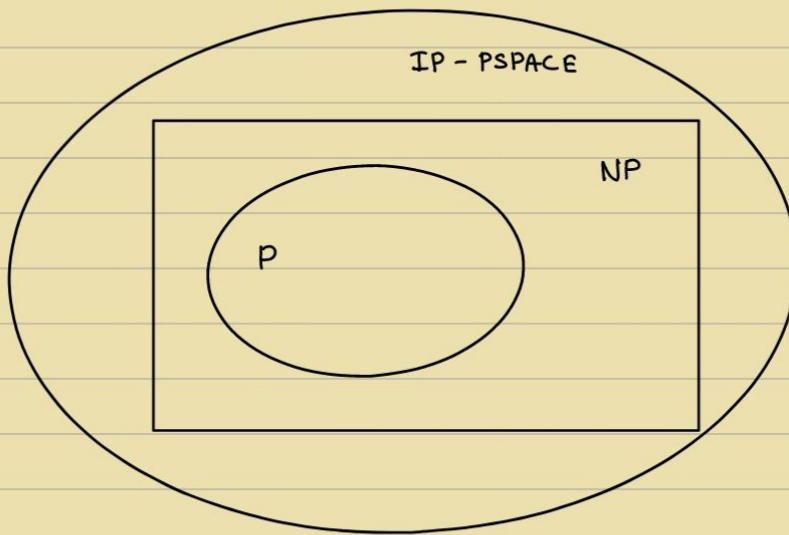
Is NP different from P?

Is NP different from IP?

- $TIME(n) = \{ L \mid L \text{ is decided by some NDTM 'M', in time } O(n) \}$

$$NP = \bigcup_{k=0}^{\infty} NTIME(n^k)$$

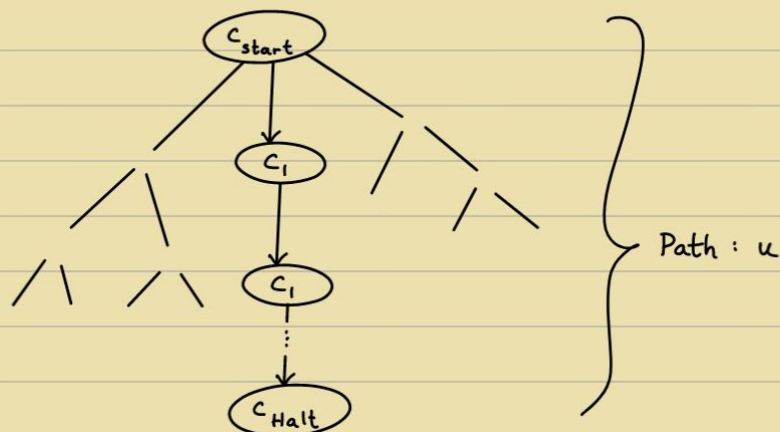
If searching for a needle in a haystack, if you're lucky you pick the needle, but the problem of finding whether you pick the needle or hay should be easy.



Verifier:

If $\exists$ path to solution: ACCEPT

If all computational paths are not accepting: REJECT



$L(M)$: M accepts x

M decides L if $L(M) = L$

Got the Needle, Then

able to verify in P

$L(V) : \{ x \mid \exists u \ V(x, u) \text{ accepts} \}$

V verifies L if $L(V) = L$

Efficient verifier : Given $x \Rightarrow$ Quickly check whether $x \in L$

A class of all polynomial time verifiers $\rightarrow P$ (Interesting)

But P : class of all Polynomial time NDTMs (Accept path according to V)

If $\exists$ NDTM for a problem $\Rightarrow \exists$ Polynomial time algo

$NP = \{ L \mid L \text{ has an efficient verifier} \}$

$\rightarrow$ In Polynomial Time

$\rightarrow$ Interest based on luck

If $\exists$ Polynomial time NDTM $\Rightarrow \exists$ Polynomial time verifier ($\exists$ a path u)

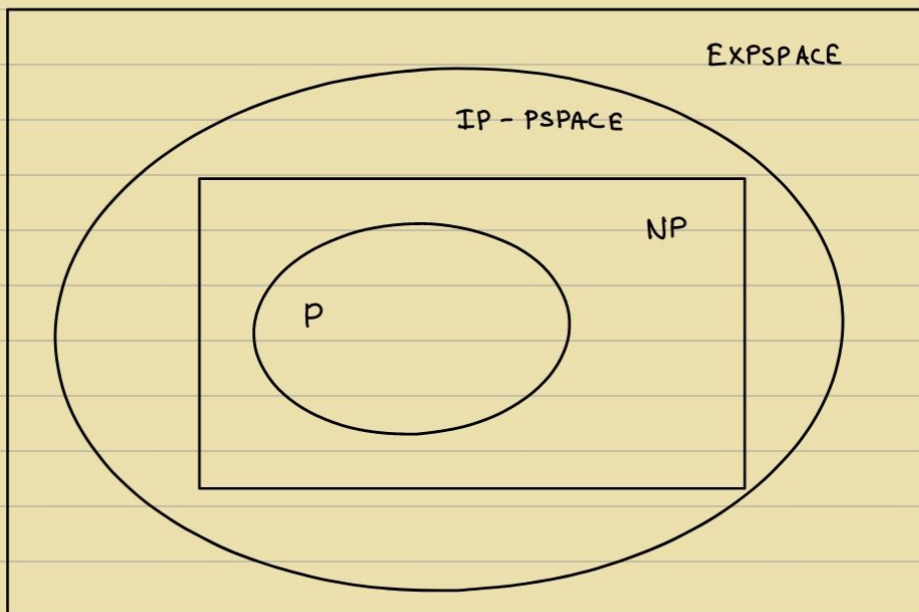
$\exists$ Polynomial time verifier $\Rightarrow \exists$ Polynomial time NDTM ($\exists u : L(V) \text{ accepts}$)

$\rightarrow$ Guess u & run

For Authorised People $\Rightarrow$ Problem is easy

For non-Authorised People $\Rightarrow$ Problem is Hard

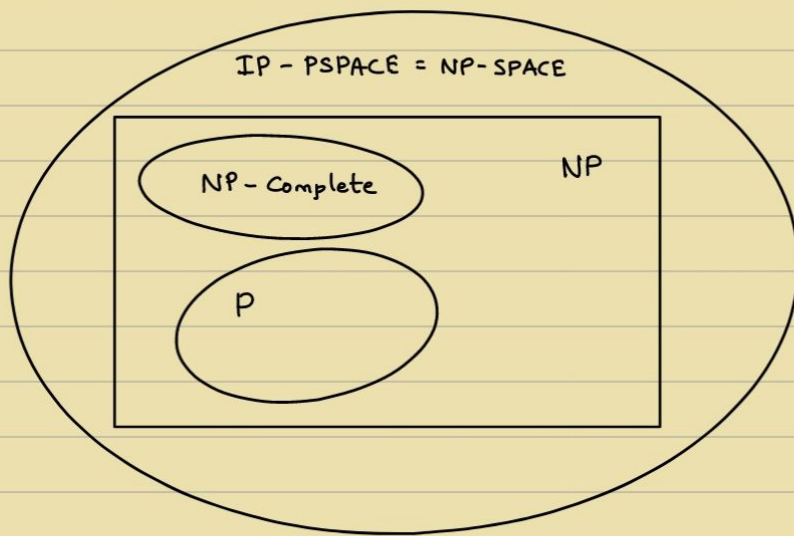
} Security



- Efficiently & easily pose hard question $\leftarrow$ Security Tenet
only when $P \neq NP$

$\therefore$ Easily Verify $\Rightarrow$ Easily decide

$\therefore$ If $\exists u \rightarrow$ Problem becomes easy (u : known)



• Reducability :

If $\exists$ Mapping Reduction

$$L_1 \leq_p L_2$$

if $\exists f$,

$$x \in L_1 \Leftrightarrow f(x) \in L_2$$

$\Rightarrow$

If $\exists$ Polynomial (time) reduction

$$L_1 \leq_p L_2$$

if $\exists f$ (Computable in Poly-time),

$$x \in L_1 \Leftrightarrow f(x) \in L_2$$

• NP-Complete :

If $\exists$ efficient solution for any NP problem, then $NP = P$

$$NPC = \{ L \in NP \mid \forall L' \in NP, L' \leq_p L \}$$

i.e. using L , you can solve every other L' .

- Theorem : Boolean satisfiability is NP-Complete.

$$SAT : \{ \phi \mid \phi \text{ is a CNF boolean formula that is not a fallacy} \}$$

(H.W : Prove $CLIQUE \in NP\text{-Complete}$)

Note : If $\exists$ an efficient solution to solve $CLIQUE$, Then every problem in NP can be efficiently solved $\Rightarrow P = NP$, i.e. all-in-one problem

Recall : General Purpose computing is possible because of existence of Universal Turing machine, which can take a TM itself as input, i.e. it's possible to build one hardware which can simulate every other hardware using the software.

NPC is used to show that $\exists$ problems that are hard, i.e. unsolvable.

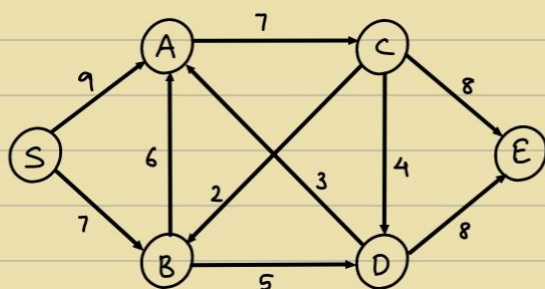
$\therefore$ NPC is used to show $CLIQUE$ is hard. If not, every problem is solvable.

$\therefore$ Approximation Algorithms, Randomisation Algorithms and Quantum Algorithms are used instead to deal with hard Problems.

6/10

→ Network Flows

$$G = (V, S)$$



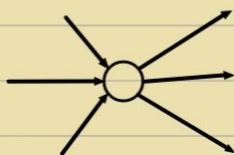
e.g. Water pipeline with flow capacities

Q) What's the maximum flow from S to E?

i.e. Objective:

$$\text{maximise } \sum_{\substack{e=(s,u) \\ e \in E}} f(e)$$

• Conserved:



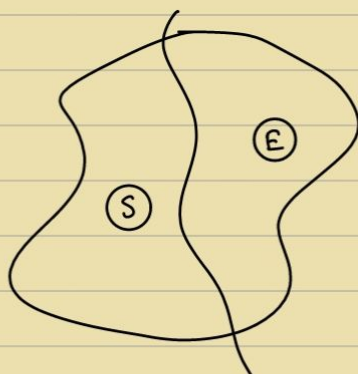
$$\text{flow, } f: E \rightarrow \mathbb{R}$$

$$\forall e \in E, f(e) \leq c(e)$$

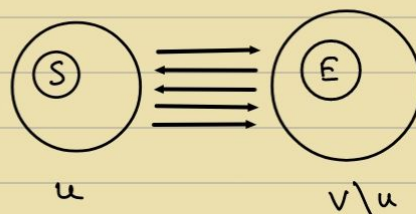
$$\text{Total-in} = \text{Total-out}$$

Theorem: Maximum Flow = Minimum Cut

$$\text{Max. flow} \leq \text{Min. cut}$$



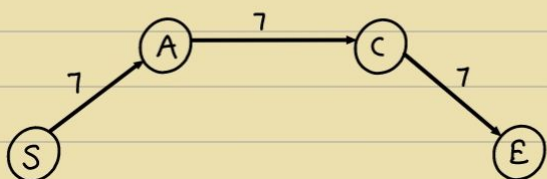
$\equiv$

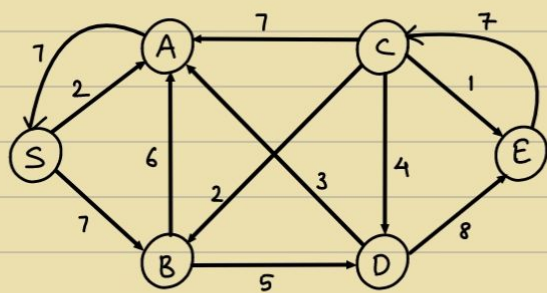


• FF Algo:

- Current Flow

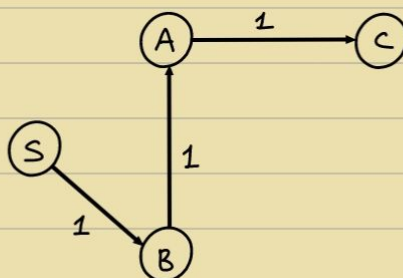
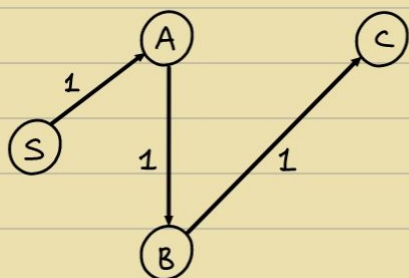
- Augment the Flow (find an augmenting path in Residual Graph)



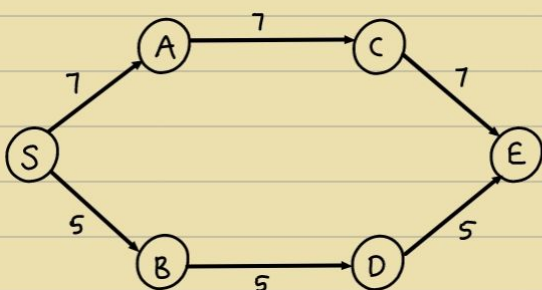
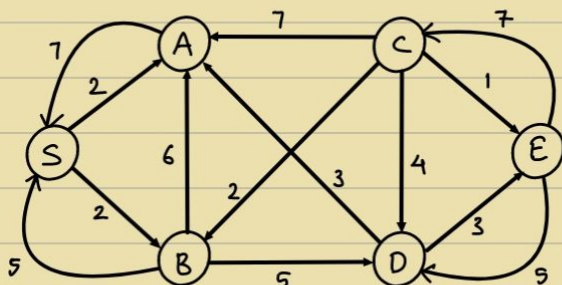


Given current flow f

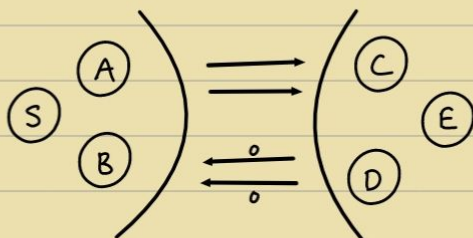
Residual Graph : $c(e) - f(e)$ (Original direction)
 (u, v) $f(e)$ (Reverse direction)



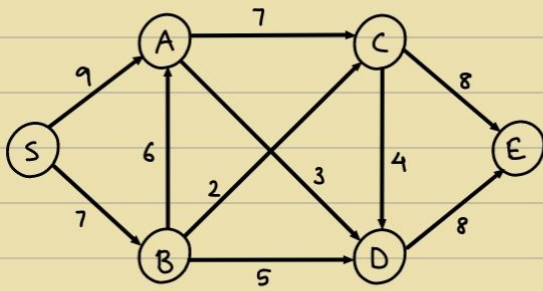
Now ,



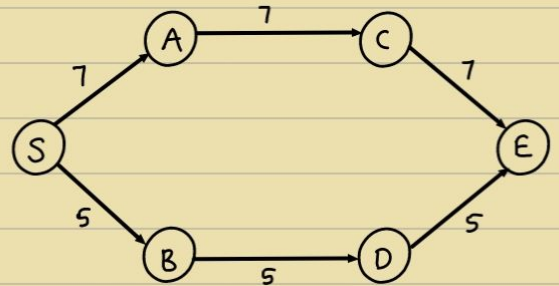
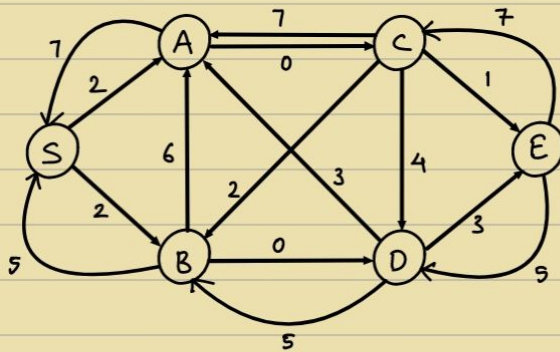
Max = 12



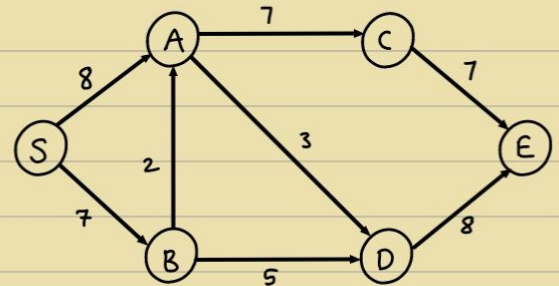
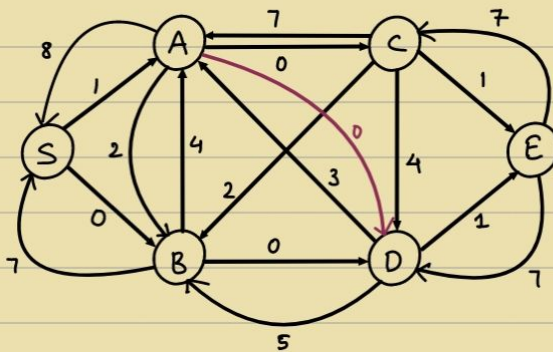
Ex.



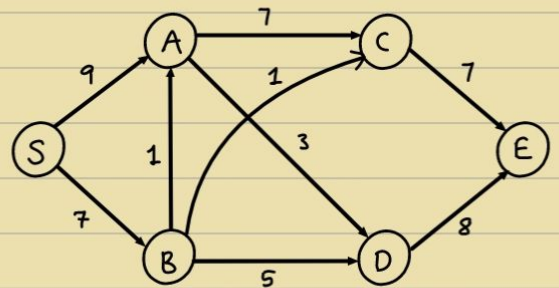
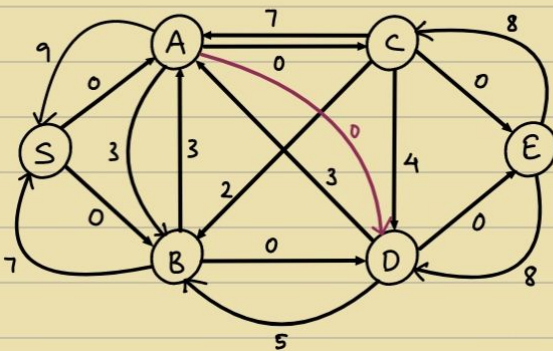
Residual graph :



3rd Augmentation :



Final Augmentation :



Max : 16

- Review:

Input: $G \equiv (V, E)$

Edge weighted

Output: $f: E \rightarrow \mathbb{R}$ [Max flow]

Conserved & Valid/feasible

Algo: Augment current flow by adding some augmenting path in/from the residual Graph

$$\begin{array}{ccc} u & \xrightarrow{c(e)-f(e)} & v \\ v & \xleftarrow{f(e)} & u \end{array}$$

- Efficient Algorithm

- But not efficient if capacity of path is extremely large.

- Simplex algorithm for linear programming $\leftarrow$ Adding 1s $\Rightarrow$ Exponential (Maybe)

But, Ford - Fulkerson algorithm $\leftarrow$ Jumps $\Rightarrow$ Polynomial

13/10

$\rightarrow$ NP-Completeness and Approximation Algorithms:

- Minimal Set Cover:

I/P: A set S $\mathcal{E}$ family of subsets of S , $F \subseteq 2^S$ (Powerset of S)

O/P: The least no. of elements that cover S (i.e. Union of S)

e.g. $S = \{1, 2, 3, 4\}$

$F = \{\{1, 2\}, \{1, 2, 3\}, \{1, 4\}, \{2, 3\}, \{2, 4\}, \{4\}\}$

Minimum set cover: 2 bcs $(\{1, 2, 3\} \cup \{4\}, \{1, 4\} \cup \{2, 3\})$

Facts:

① Minimum Set Cover is NP-Hard.

(How to show this?)

② There is a greedy strategy that approximates the answer within a factor of $\log |S|$. $\leftarrow$ Approximation Ratio

i.e. Computed answer $\leq \log |S|$ [OPTIMUM]

Greedy Algorithm of min-set cover :

Choose maximum |element| set and keep choosing max-sets till all elements covered.

for above e.g. ① $\{1, 2, 3\}$

② $\{1, 2, 3\} \{4\} \rightarrow 2$, Here it's optimal

But it need not be like this always

Greedy Algo :

Always choose the next element of $\mathcal{I}$ that covers the maximum number of not-yet-covered elements of S .

Theorem :

Approximation ratio is $\log |S|$
(of greedy not-cover)

i.e. optimal answer ($\#$ of sets that cover S) $\leq \log |S|$

To solve : $t \leq \log |S| \cdot k$ [OPTIMUM]

Proof :

Let n_i be the no. of uncovered elements in S after i iterations of the greedy algo

$n_0 = S$		After i iterations ,
$n_t < 1$		n_i : remain uncovered
		: $\exists k$ sets that cover it

Using PHP :

$\exists$ a set that covers $\geq \frac{n_i}{k}$

$$\Rightarrow n_{i+1} \leq n_i - \frac{n_i}{k}$$

$$\Rightarrow n_{i+1} \leq n_i \left(1 - \frac{1}{k}\right)$$

$$n_i \leq n_0 \left(1 - \frac{1}{k}\right)$$

$$\Rightarrow n_i \leq |S| \left(1 - \frac{1}{k}\right)$$

$$\Rightarrow 1-x \leq e^{-x} \quad [0 < x < 1]$$

$$\Rightarrow \left(1 - \frac{1}{k}\right) \leq e^{-1/k}$$

$$\Rightarrow n_i \leq |S| \cdot e^{-1/k}$$

$$② \quad i = k \cdot \ln |S| \Rightarrow n_{k \cdot \ln |S|} \leq e^{-\frac{k \ln |S|}{k}} = 1$$

$$\therefore t \leq \ln |S| \cdot k$$

after $t = \ln |S| \cdot k$ iterations, greedy algo terminates.

→ SAT:

$SAT = \{ \phi \mid \phi \text{ is a satisfiable CNF Boolean formula} \}$

$$\phi = (x \vee x \vee \dots \vee x) \wedge (\dots) \wedge (\dots)$$

Cook - Levis Theorem :

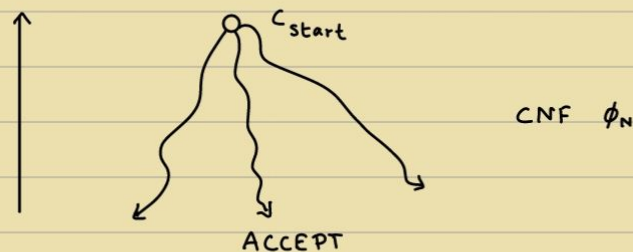
SAT is NP - Complete .

i.e. (a) $SAT \in NP$

(b) $\forall L \in NP \text{ s.t. } L \leq_p SAT$

$\left\{ \begin{array}{l} \text{Karp Reduction :} \\ \rightarrow \text{Polytime mapping reduction} \end{array} \right\}$

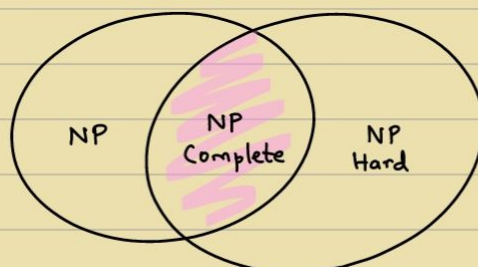
Note : $L \in NP$ means that $\exists$ NDTM 'N' that decides L in Poly time.



ϕ_N is satisfiable iff N has an accepting path.

$\{ \text{NDTM form} \longrightarrow \text{Boolean form : Reduction} \}$

$A \leq_p B$ if $\exists$ poly-time computable f s.t. $\forall x \in \{0,1\}^*$,
 $x \in A \Rightarrow f(x) \in B$



Th: CLIQUE is NP-Complete

Proof: Recall $\text{CLIQUE} \in \text{NP}$

$$\text{CLIQUE} = \{ \langle G, k \rangle \mid G \text{ has a clique of size } k \}$$

To show: $\forall L \in NP, L \leq_p \text{CLIQUE}$ (CLIQUE is NP-Hard)

$$\text{SAT} \leq_P \text{CLIQUE}$$

$$\text{SAT} \leq_p \underbrace{3\text{-SAT}}$$

$[2\text{-SAT is in } P]$

$$\rightarrow = \{ \phi \mid \phi \text{ is a satisfiable 3-CNF} \}$$

3 CNF $\phi = (\cdot v \cdot v \cdot) \wedge (\cdot v \cdot v \cdot) \wedge \dots$

i.e. exactly 3 literals per clause

e.g. $(x_1 \vee \bar{x}_2 \vee x_3) \wedge (\bar{x}_1 \vee x_2 \vee \bar{x}_3) \wedge \dots$

if there are more than 3, then reduce to 3 CNF:

i.e. $(x_1 \vee x_2 \vee x_3 \vee x_4)$

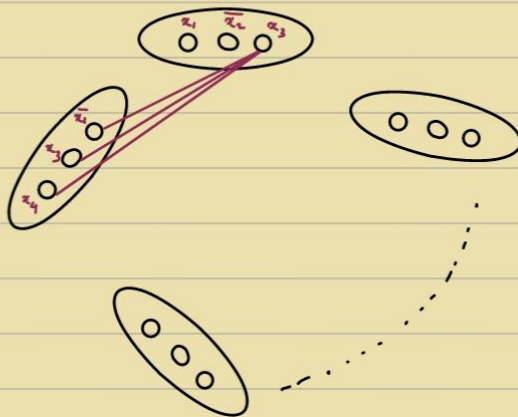
$$= (x_1 \vee x_2 \vee x_5) \wedge (x_3 \vee x_4 \vee \overline{x_5})$$

Given 3 CNF : $\phi = (\cdot \vee \vee \cdot) \wedge () \wedge () \dots \dots \wedge ()$

n variables

2 clauses

Construct a graph G_0 :



32 vertices

Add all non-contradictory edges across clauses.

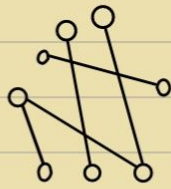
i.e. (a) Don't add edge in
the same clause/node

(b) $\bar{x}_i$ and x_i should not be added

Does G_n have a clique of size l ?

Satisfying assignment

$x_1, \dots, \bar{x}_l \rightarrow l$ literals

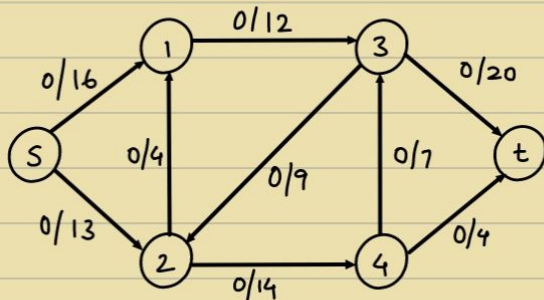
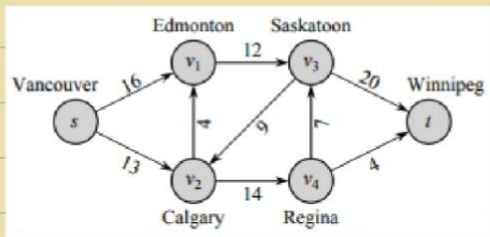


l vertices

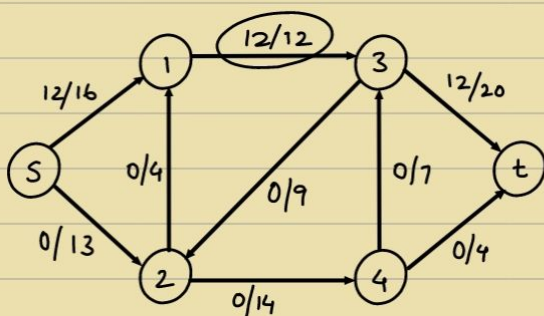
$\therefore$ CLIQUE is NP Hard.

$\therefore$ CLIQUE is NP complete. ($\because$ CLIQUE is also in NP)

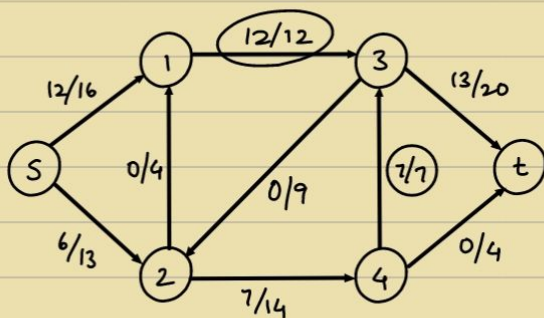
(4)



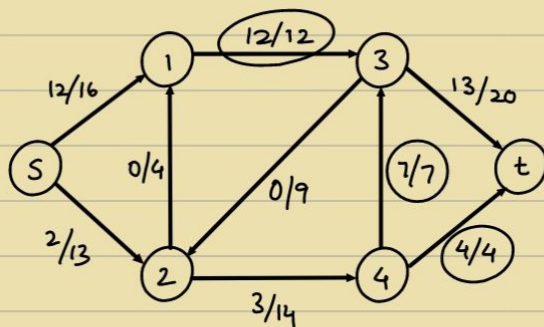
$s \rightarrow 1 \rightarrow 3 \rightarrow t : 12$



$s \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow t : 7$



$s \rightarrow 2 \rightarrow 4 \rightarrow t : 4$



Maximum flow : $12 + 7 + 4 = 23$

19)

Clique Decision Problem (CDP)

Sat $\propto$ CDP

x_1, x_2, x_3

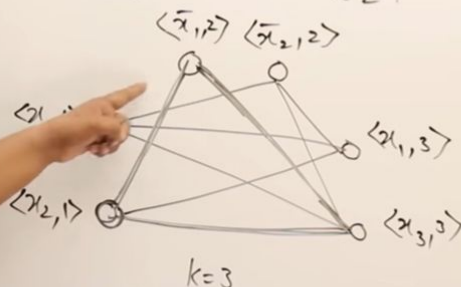
$K=3$

$$F = (\overset{0}{x_1} \vee \overset{1}{x_2}) \wedge (\overset{1}{x_1} \vee \overset{0}{x_2}) \wedge (\overset{0}{x_1} \vee \overset{1}{x_3}) = 1$$

$C_1=1$

$C_2=1$

$C_3=1$



$x_1 \quad x_2 \quad x_3$
0 1 1

20)

Independent Set Problem Given a graph G and an integer K , does G contain an independent set of size K ?

Proposition 8. The Independent Set Problem can be reduced to the Vertex Cover Problem in polynomial time.

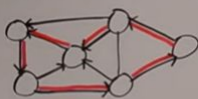
Proof. Let G be a graph and integer K be an instance of the independent set problem. Let n be the number of vertices of G and m be the number of edges.

Graph G has an independent set of size K if and only if G has a vertex cover of size $n - K$.

The mapping (G, K) to $(G, n - K)$ is a reduction of the independent set Problem to the vertex cover Problem and copying G takes $\Theta(n + m)$ time which is a polynomial in n and m . $\square$

21)

Defn: A Hamiltonian Cycle in graph G is a simple cycle that visits every vertex.



HC problem: input = graph G
output = whether G has a Ham. cyc.

Thm: HC is NP-complete

1) HC $\in$ NP

• cert = order in which vertices are visited in the cycle

• verifier: check all n vertices appear exactly once, and each is adjacent to the next.

• poly size/poly time $\checkmark$

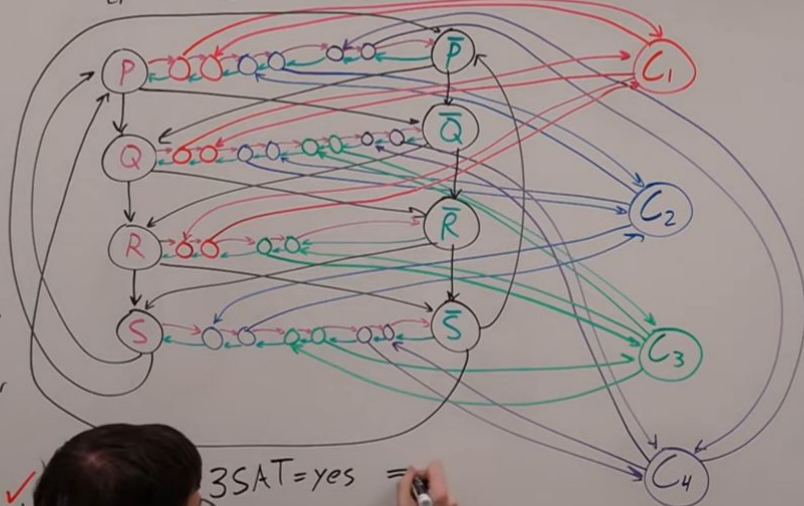
• G has HC $\Leftrightarrow \exists$ cert that verifier accepts $\checkmark$

2) $3SAT \leq_p HC$

$(p_1 \vee q_1 \vee r_1) \wedge (\bar{p}_1 \vee q_1 \vee \bar{s}_1) \wedge (\bar{q}_1 \vee r_1 \vee \bar{s}_1) \wedge (\bar{p}_1 \vee \bar{q}_1 \vee s_1)$

3SAT $\xrightarrow{\text{transform}}$ HC

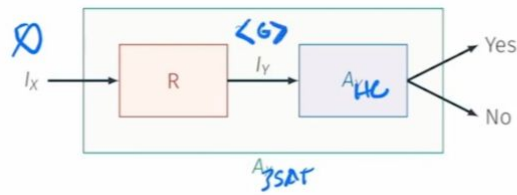
$G=(V,E)$



3SAT = yes =

(OR)

- Directed Hamiltonian Cycle is in NP: exercise
- **Hardness:** We will show $3\text{-SAT} \leq_P \text{Directed Hamiltonian Cycle}$

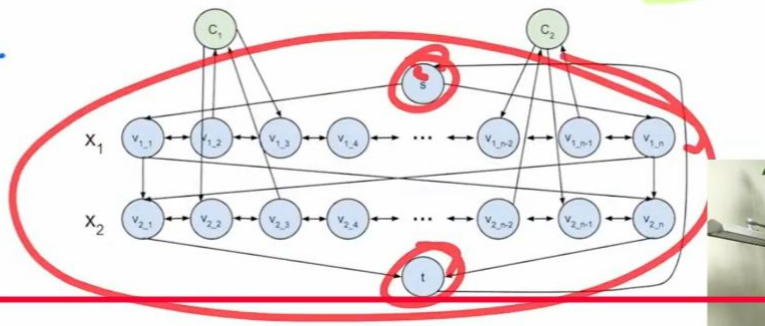


How do we handle clauses?

$$f(x_1, x_2) = (x_1 \vee \bar{x}_2) \wedge (\bar{x}_1 \vee x_2)$$

What if the expression has multiple clauses:

$\begin{matrix} 1 \rightarrow \\ 0 \leftarrow \end{matrix}$



$\begin{matrix} 0 & 0 \\ 0 & 1 & c_1 \\ 1 & 0 & c_2 \\ 1 & 1 \end{matrix}$

23) **Vertex Cover Problem** Given a graph G and an integer M , does G have a vertex cover of size M ?

Proposition 10. The Vertex Cover Problem can be reduced to the Set Cover Problem in polynomial time.

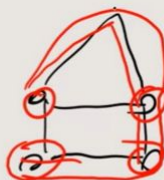
Proof. Let graph $G = (V, E)$ and integer M be an instance of the vertex cover problem. Let $U = E$ and let $S_i = \{e_j : v_i \text{ is incident on } e_j\}$ and C be the collection $S_1, S_2, \dots, S_n$. Graph G has a vertex cover of size M if and only if U has a set cover of size M .

Mapping (G, M) to (U, C, M) is a reduction of the vertex cover problem to the set cover problem and constructing U and C takes $O(mn)$ time which is a polynomial in m and n . $\square$

22)

① $LP \in NP$ (easy)check path exists if someone gave you a path
such

in polynomial time.

② LP is NP -hardHamiltonian path $\leq LP$ Hamiltonian path: $G = (V, E)$ there is a path connecting all vertex without
repeating ~~vertex~~
vertices

pf:

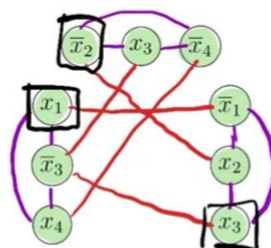
Given $G = (V, E)$ $\rightarrow G = (V, E), K = |V| - 1$ 

IS :

graph has an independent set, then φ is satisfiable,

$$\varphi = (x_1 \vee \bar{x}_3 \vee x_4) \wedge (\bar{x}_2 \vee x_3 \vee \bar{x}_4) \wedge (\bar{x}_1 \vee x_2 \vee x_3)$$

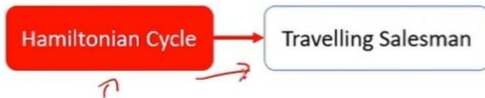
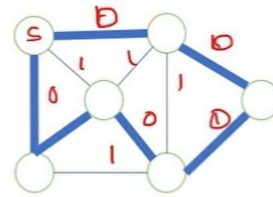
1. Create a vertex for each literal.
2. Connect each literal to the other two literals in the same clause.
3. Connect each literal x_i to $\bar{x}_i$.



TSP :

Travelling Salesman $\in$ NP

Given a sequence of vertices. The verification algorithm checks that this sequence contains each vertex exactly once, sums up the edge costs, and checks whether the sum is at most k .



Hamiltonian Cycle & Travelling Salesman

Let $G=(V, E)$ is an instance of Hamiltonian Cycle. An instance of TSP is constructed as, Form a complete graph $G'=(V, E')$. Where $E'=\{(i, j): i, j \in V \text{ and } i \neq j\}$

Cost is defined as

$$c(i,j) = \begin{cases} 0 & \text{if } (i,j) \in E, \\ 1 & \text{if } (i,j) \notin E \end{cases}$$

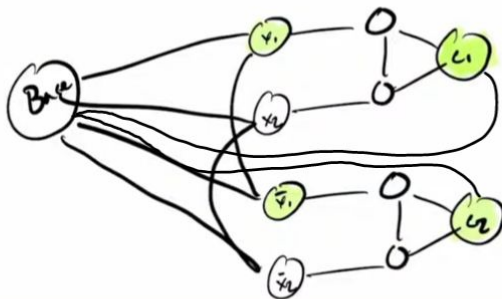
The graph G has a Hamiltonian cycle if and only if graph G' has a tour of cost at most 0.

TSP is Hamiltonian

3 Colouring

Next we need a gadget that assigns literals. Our previously constructed gadget assumes:

- All literals are either red or green.
- Need to limit graph so only x_1 or $\overline{x_1}$ is green. Other must be red

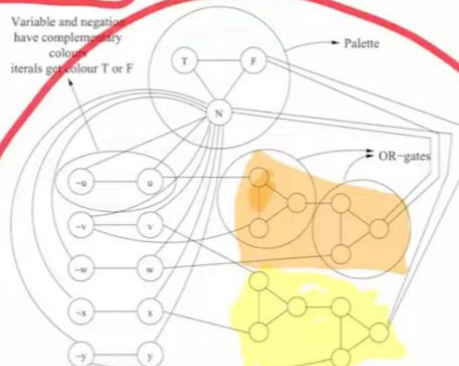


$$\varphi = (x_1, v_{p_2}) \wedge (\bar{x}_1, v_{\bar{p}_2})$$



Example

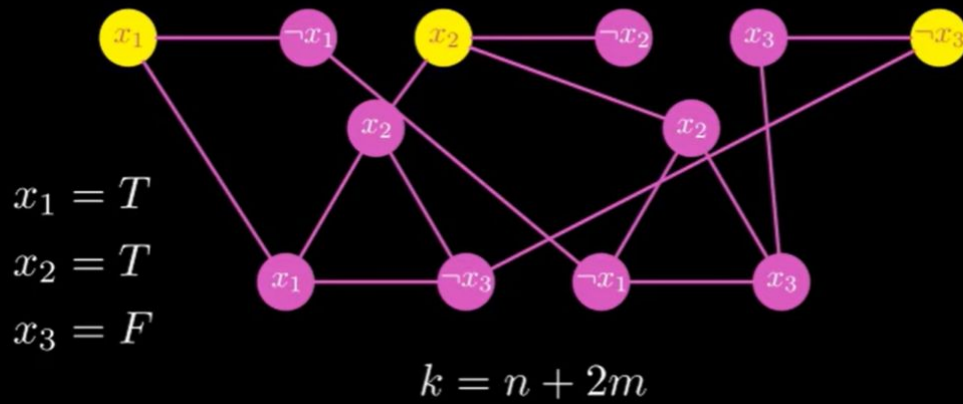
$$\varphi = (u \vee \neg v \vee w) \wedge (v \vee x \vee \neg y)$$



Vertex Cover:

Reduction from 3-SAT to Vertex Cover

$$(x_1 \vee x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee x_3)$$



Approximation Algo's :

Knapsack :

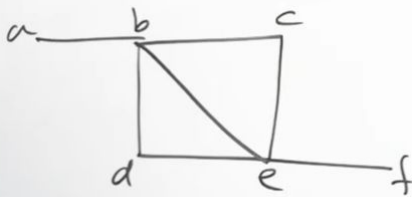
Approximation Schemes -

- This algorithm generates all subsets of k items or less & for each one that fits into knapsack it adds the remaining items as greedy alg would do (ie, in non increasing order of v/w ratio).
- The subset of highest value obtained in this fashion is returned as alg op.

$$\frac{f(S^*)}{f(S_a^k)} \leq 1 + \frac{1}{k} \rightarrow \text{Accuracy Ratio.}$$

Vertex Cover :

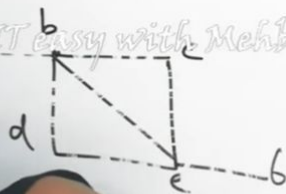
q)



$$\{b, e\} = 2$$

← 2- approximation Algo

$$\{a, b, d, e\} = 4$$



TSP :

$$\max \left(\frac{C^*}{C}, \frac{C}{C^*} \right) = f(n)$$

$$\frac{8}{4} = 2$$

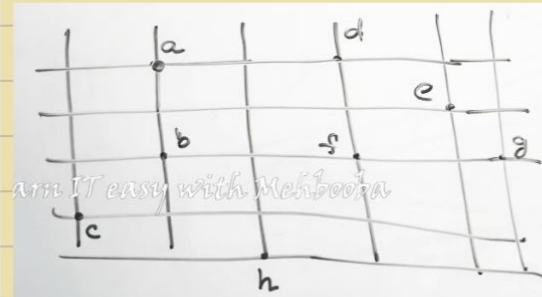
$$f(n) > 1$$

$$\epsilon_p > 0$$

$$1 + \epsilon$$

$$F = 1.66$$

$$Nf = 2$$



Prims Algo - MST

Preorder Traversal

Hamiltonian Cycle (Preorder)

Minimum Partitioning:

12 10 9 7 4 3 2

$$A = 12 + 10 + 2 = 24$$

$$B = 9 + 7 + 4 + 3 = 23$$

Learn IT easy with Mehbooba

$$\begin{array}{r} A \\ 0+12 \\ +7 \\ +4 \\ \hline \end{array}$$

$$\begin{array}{r} B \\ 0+10 \\ +9 \\ +3 \\ +2 \\ \hline \end{array}$$

$$\begin{array}{r} 1+8 \\ \hline \end{array}$$